

Make Your Code Faster with Data Parallelism

Daniel Lemire, professor

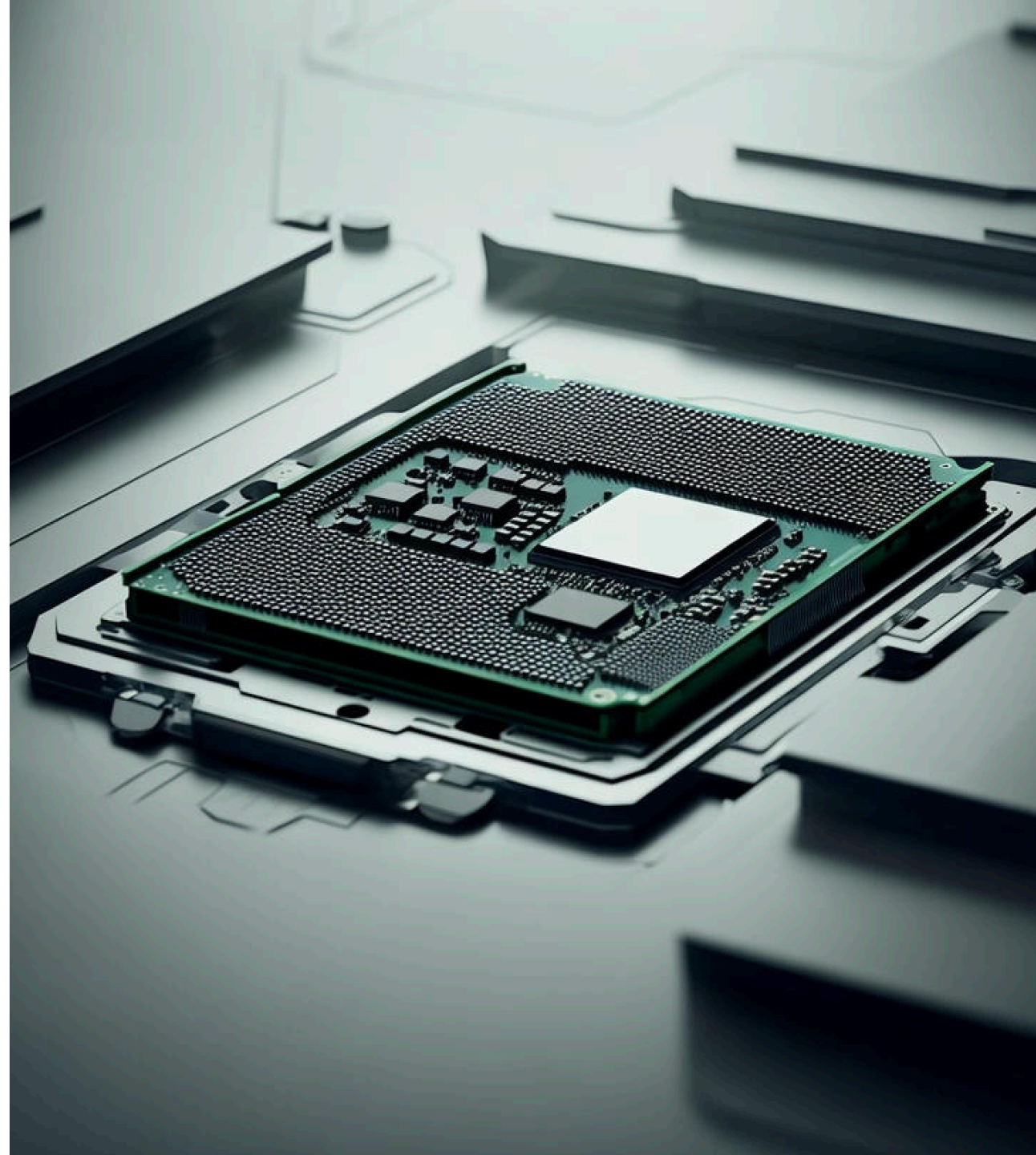
Université du Québec (TÉLUQ)

Montréal 🇨🇦

blog: <https://lemire.me>

X: [@lemire](#)

GitHub: <https://github.com/lemire/>



Where I am coming from

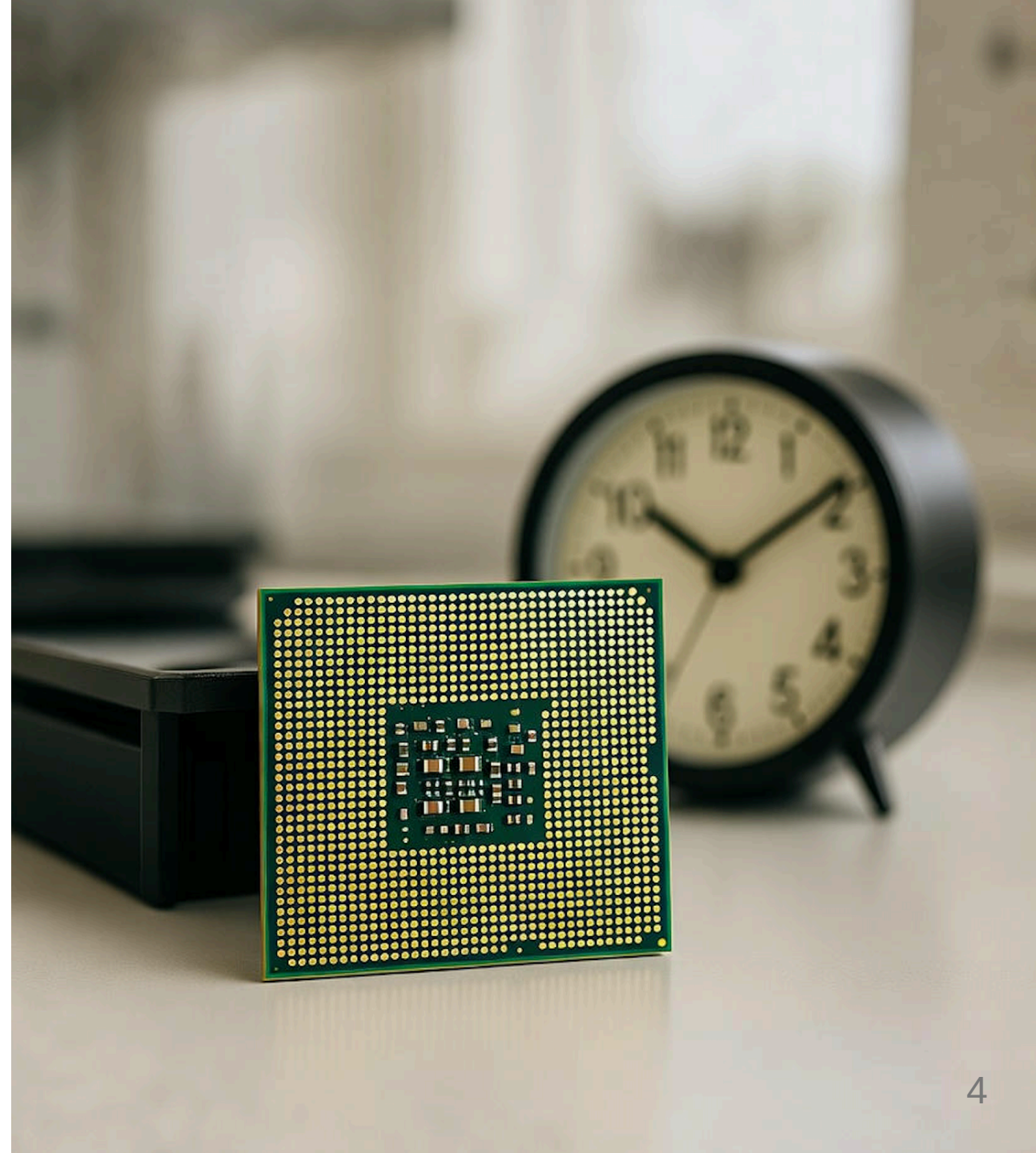
- Author of libraries you are probably running right now: **simdjson**, **simdutf**, **fast_float**, **Roaring Bitmaps**.
- Shipped inside .NET, Rust, Go, Node.js, Bun, Deno, Google Chrome, Safari, ClickHouse.
- Node.js core contributor (C++ review, performance, security).
- Editor, *Software: Practice and Experience* (Wiley, founded 1971).

Part 1

Parallelism for the win

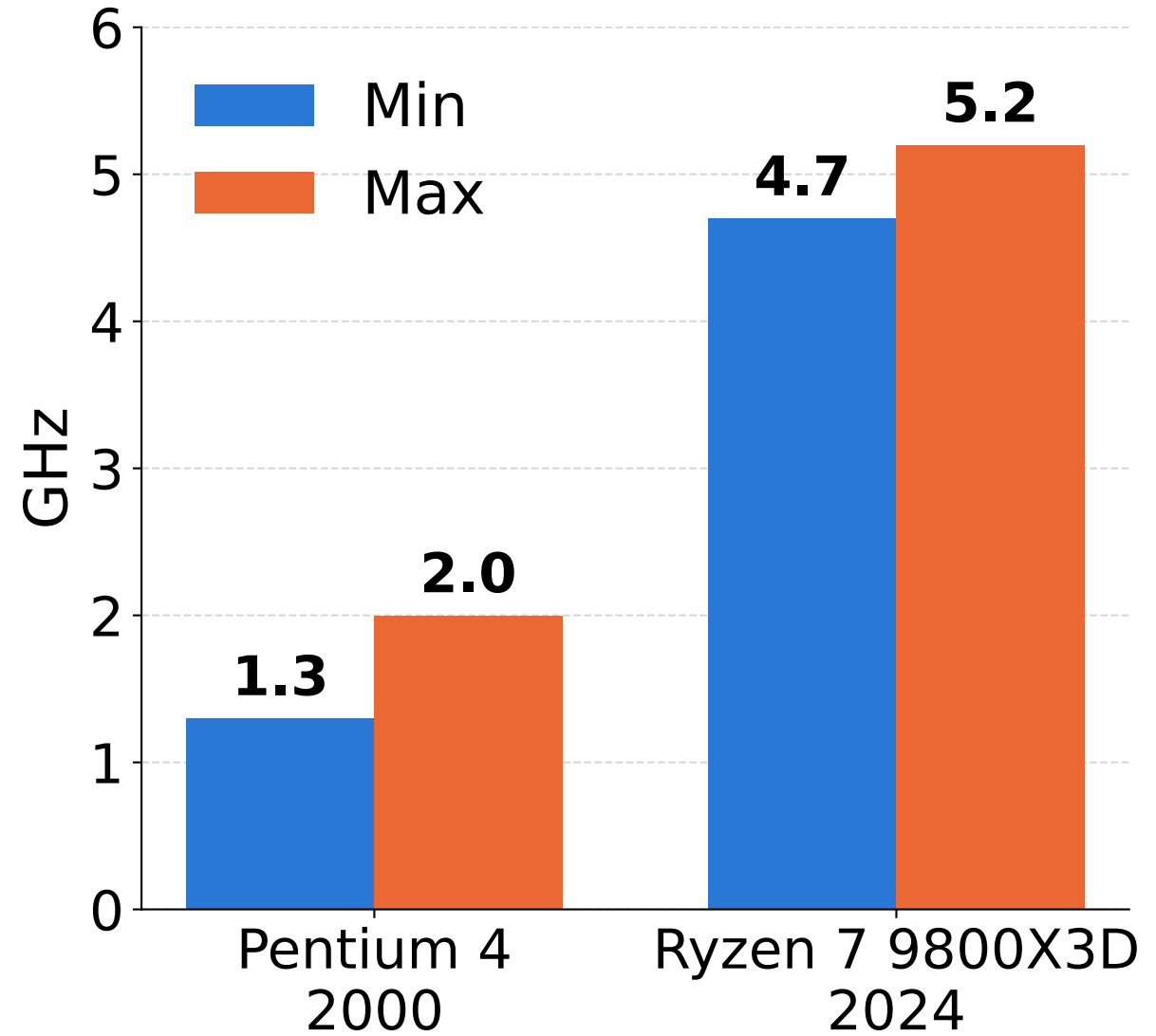
Some numbers

- Time is discrete: the clock cycle
- Processors: ~ 4 GHz
- One cycle is 0.25 nanoseconds
- Light travels 7.5 cm per cycle
- **One byte per cycle is only 4 GB/s**



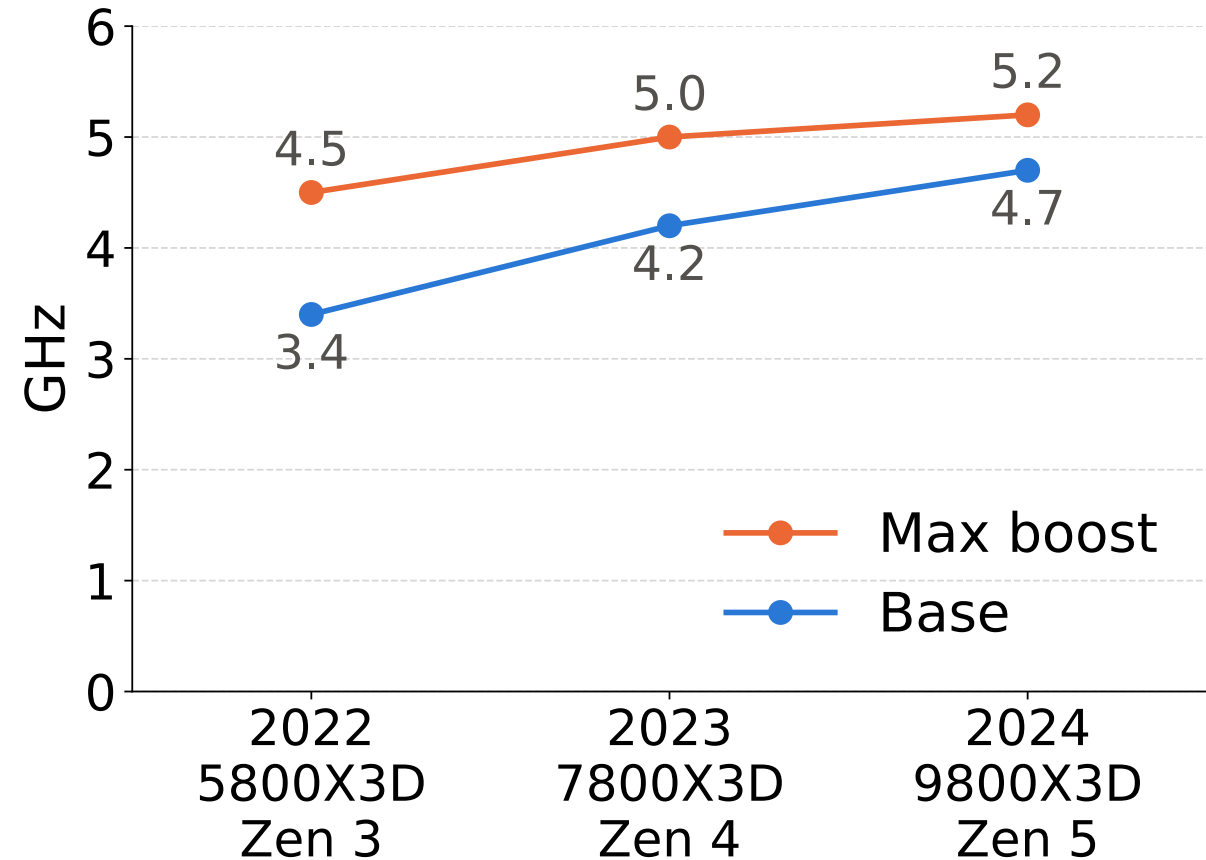
Clock frequency: 25 years

- Pentium 4 (2000): 1.3 to 2 GHz
- AMD Ryzen 7 9800X3D (2024): 4.7 to 5.2 GHz
- **About 2.5× in 25 years**

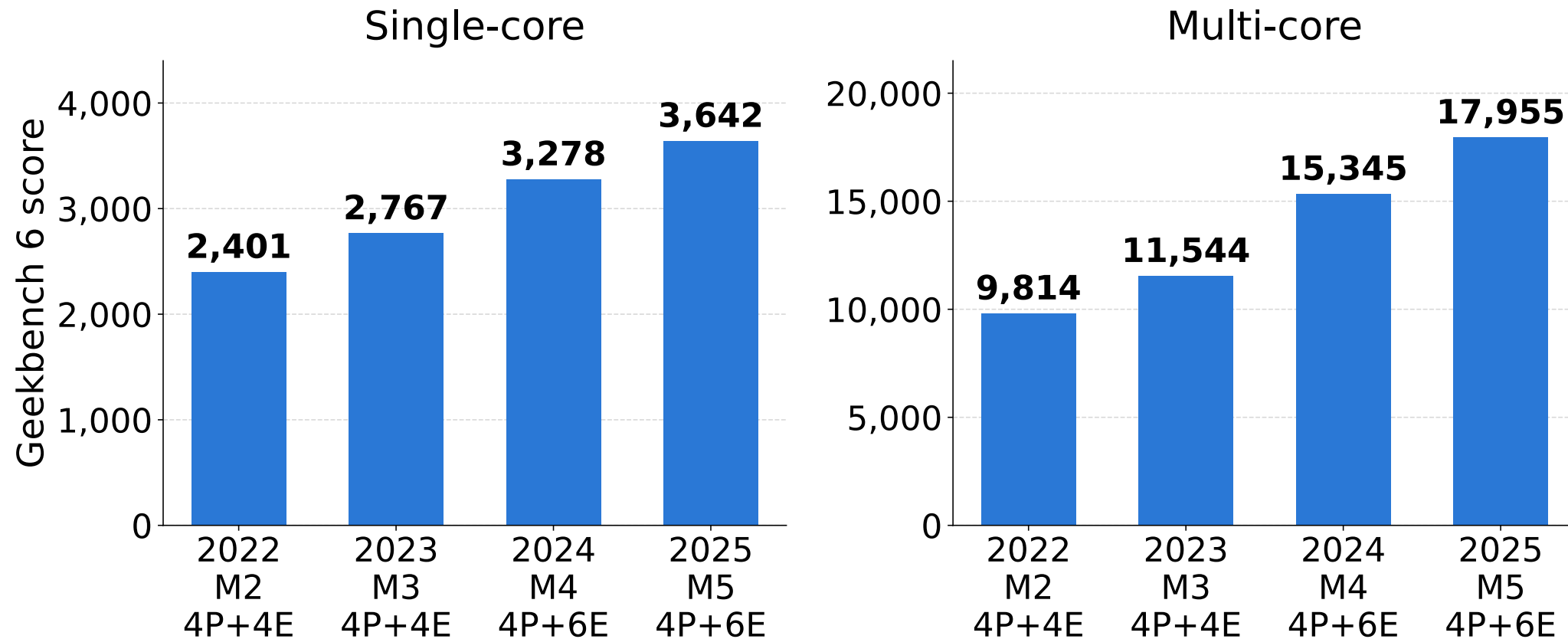


Clock frequency: AMD Ryzen 7, 2022– 2024

- Up 15% is two years

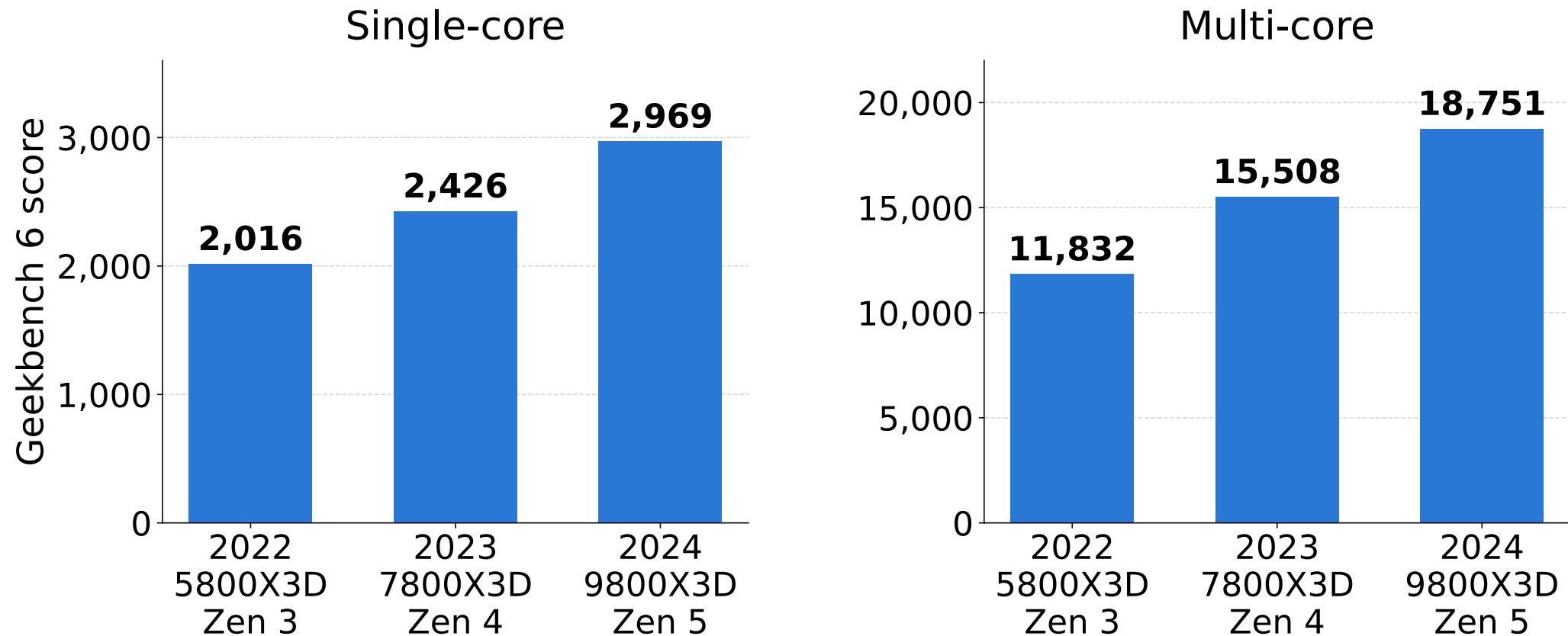


Geekbench 6: Apple M2–M5, 2022–2025



Single-core: **+52%**. Multi-core: **+83%**. Same 4 P-cores.

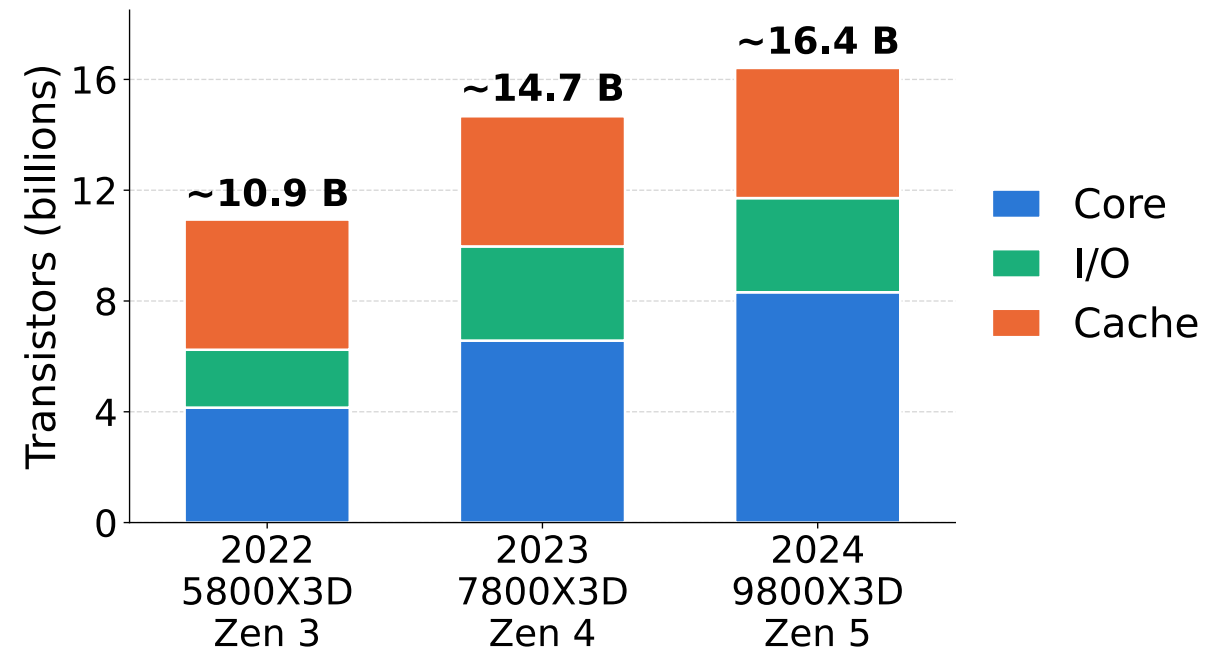
Geekbench 6: AMD Ryzen 7, 2022–2024



Single-core: **+47%**. Multi-core: **+58%**. Same 8 cores.

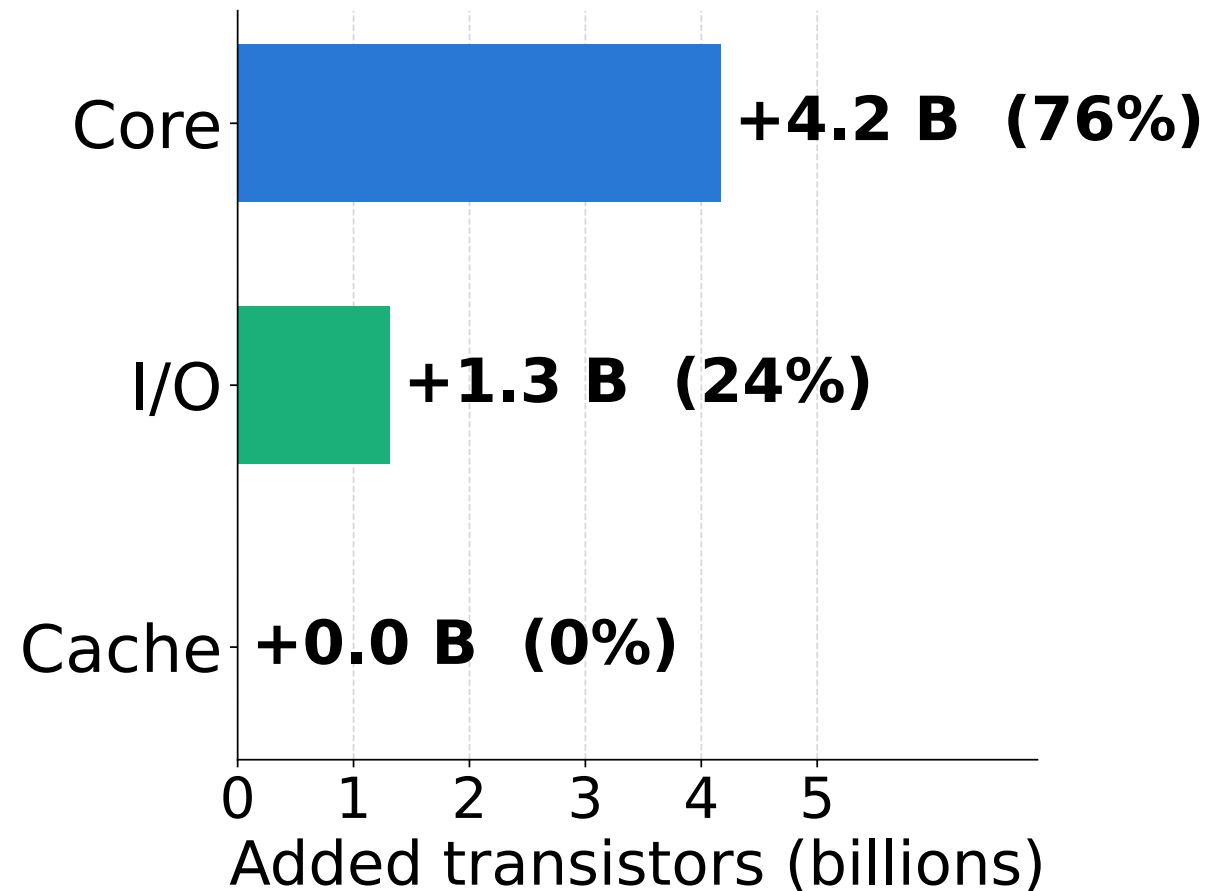
Transistors: AMD Ryzen 7, 2022–2024

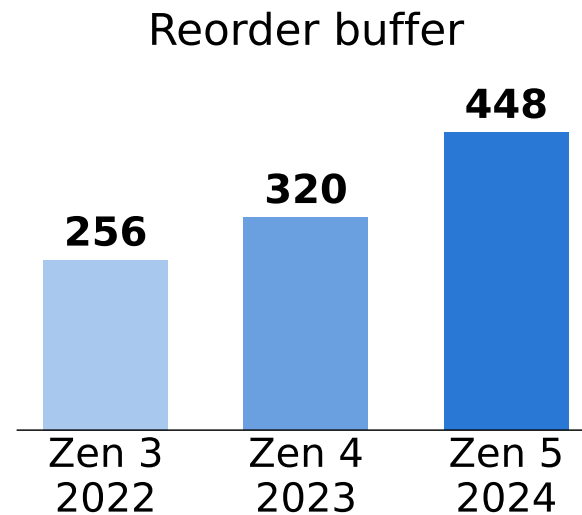
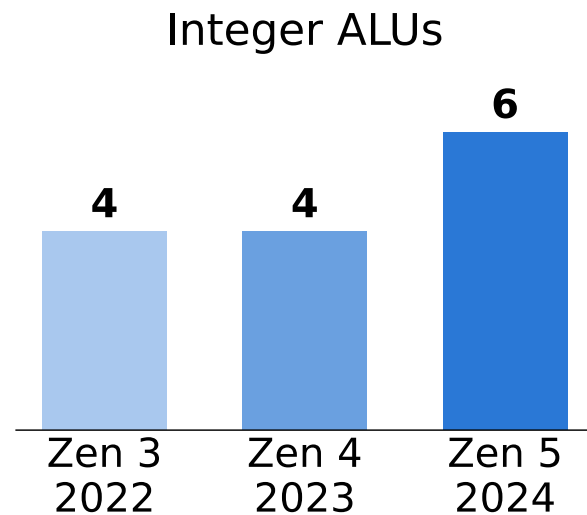
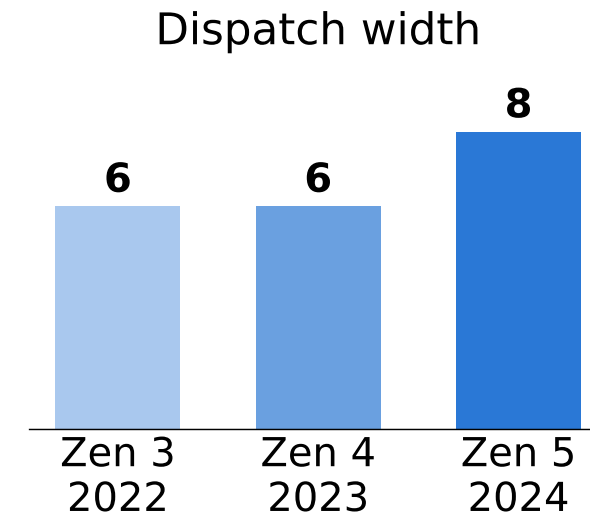
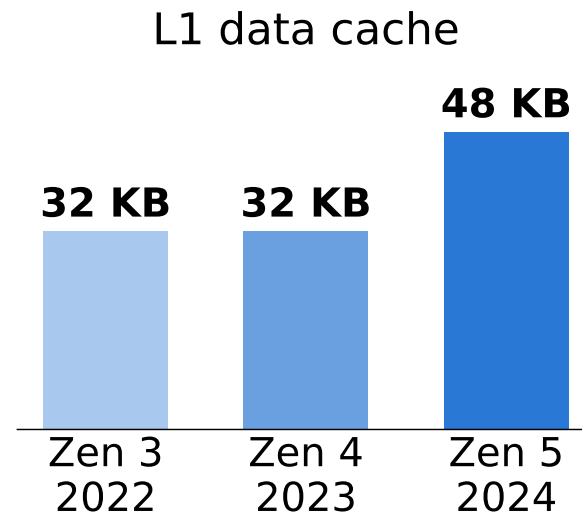
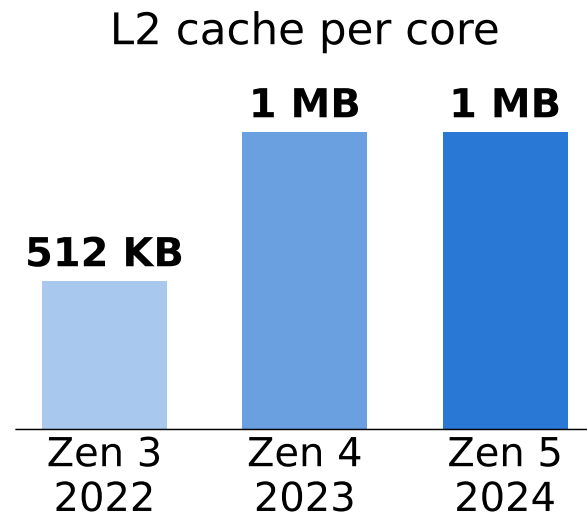
- 8 cores
- Up 50% is two years



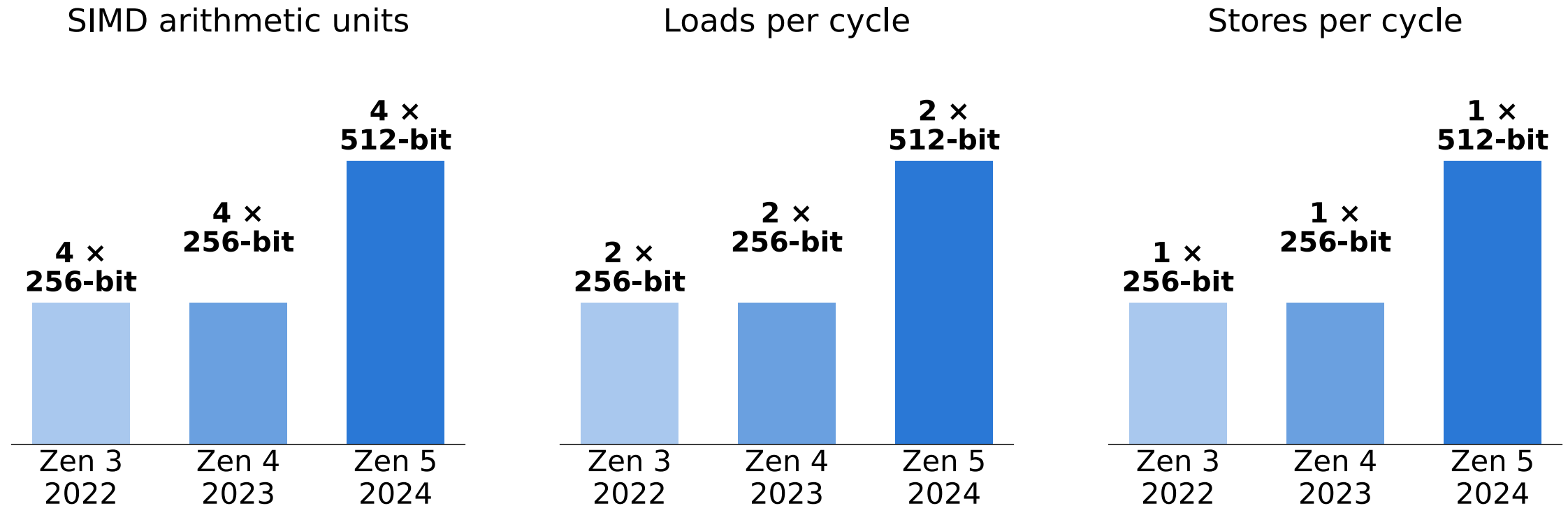
+5.5 B transistors, 2022–2024

- Same 8 cores
- **76%** went into the core die
- 24% into the I/O die
- Cache die unchanged



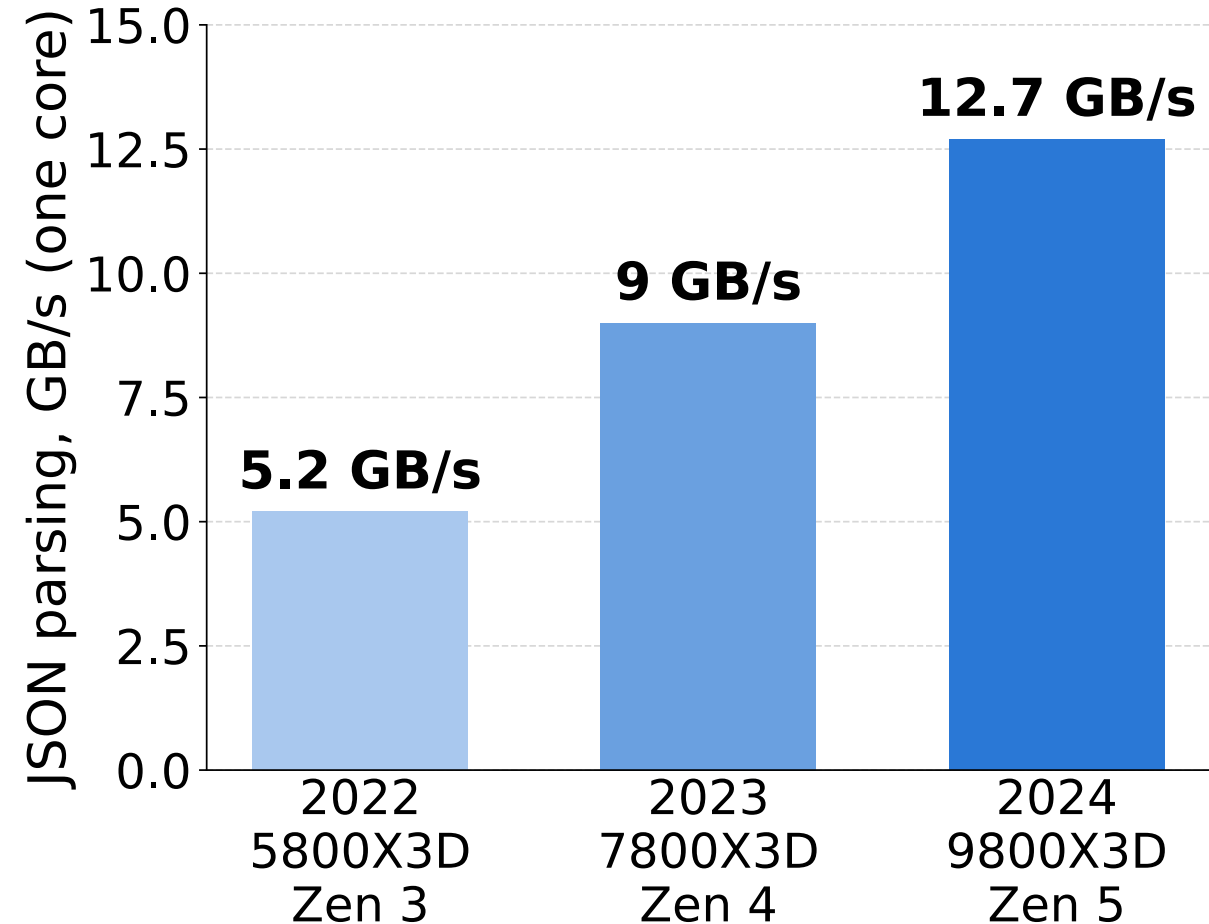


What changed in the core: SIMD (same 8 cores)



Same software, three years apart

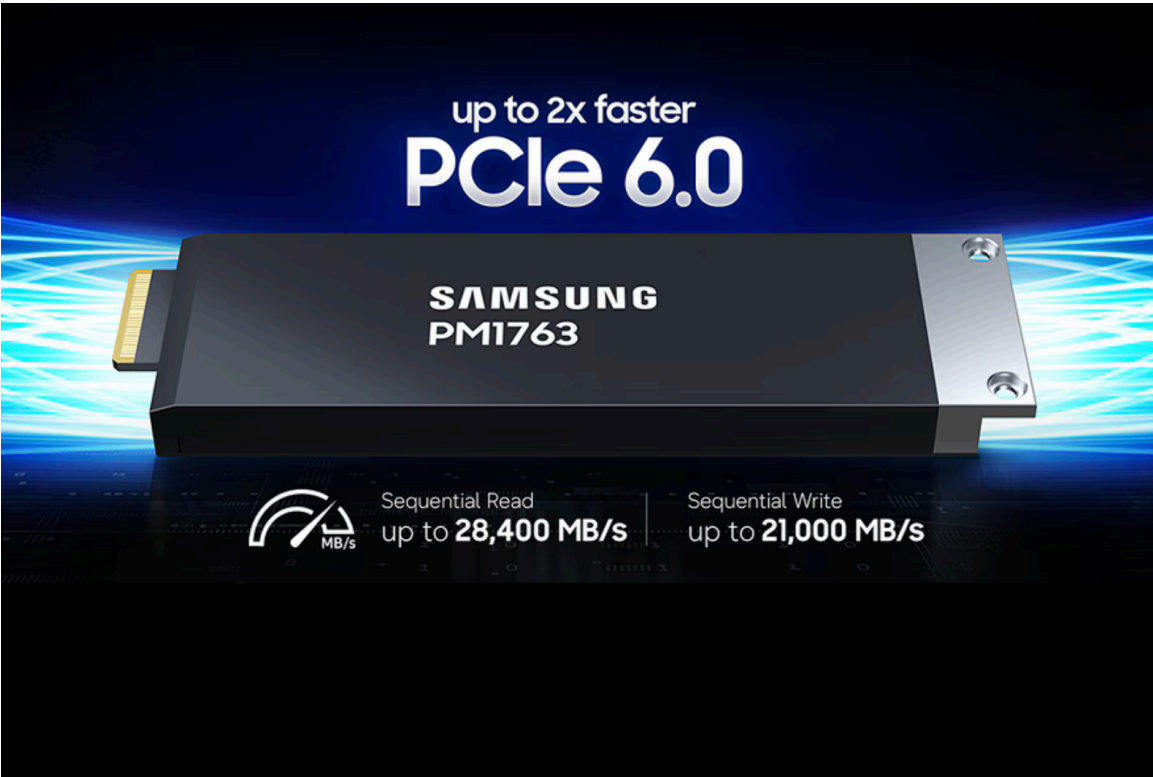
- simdjson, PartialTweets benchmark, one core
- Same code: **2.4×** from 2022 to 2024
- The library is designed for data parallelism
- Same \$



Disk at gigabytes per second

device	year	PCI	read speed
PS5	2020	4.0	5 GB/s
WD Black SN8100	2025	5.0	14.9 GB/s
Samsung PM1763	2026	6.0	28.4 GB/s

|

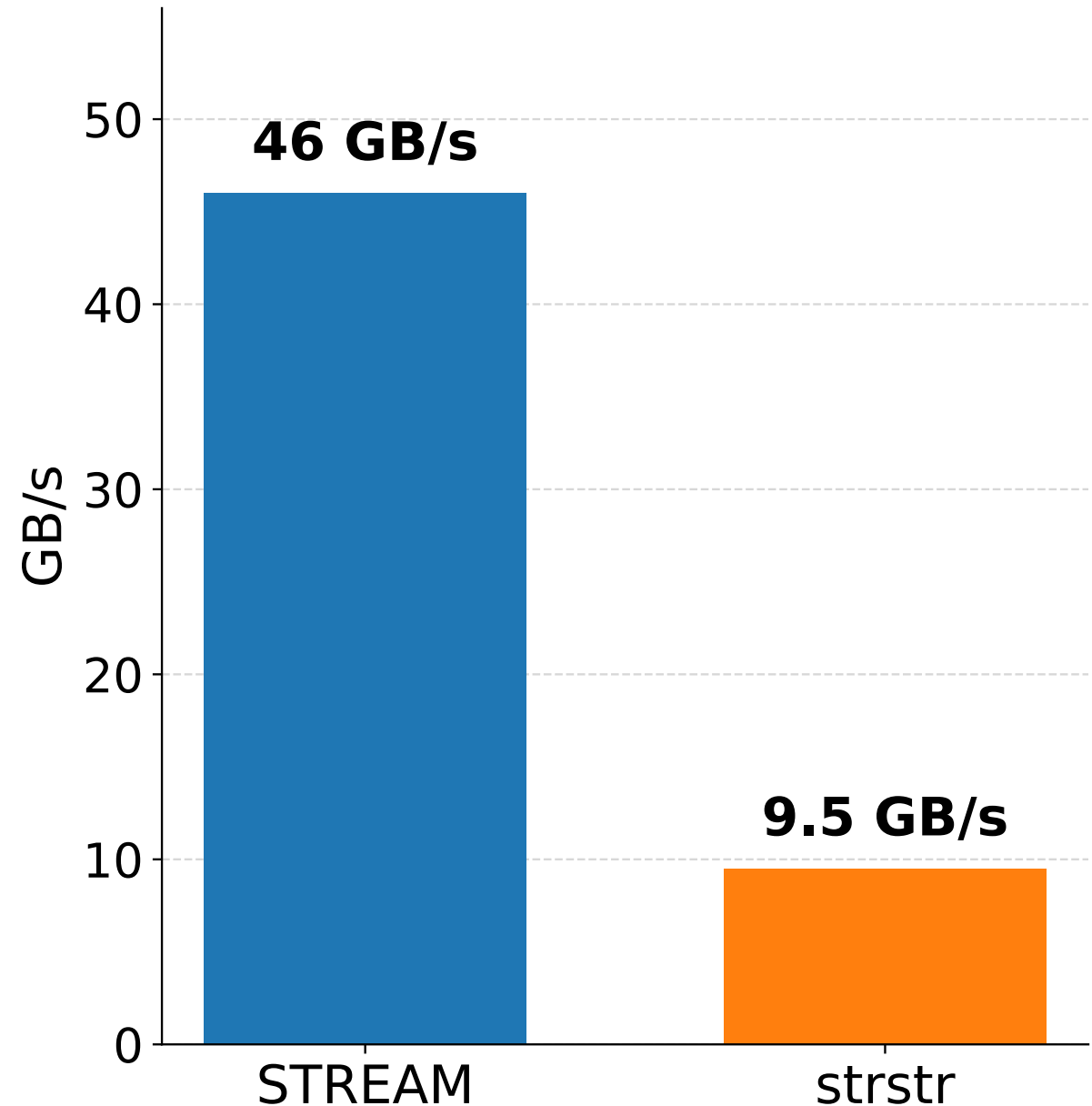


You are CPU bound.

More than you know.

Zen 5 AWS (EPYC 9R45, c8a)

- STREAM single thread bandwidth: 46 GB/s
- `strstr`, 32-byte needle: 9.5 GB/s



Data-level parallelism

The rest of this talk

SIMD (Single Instruction, Multiple Data)

- Process 16, 32 or 64 bytes with **one** instruction
- Supported on every modern CPU — your phone included
- Not a niche feature: it is most of the silicon area of a modern core

Scalar: 1 instruction, 1 result

a  +  =  × 8 instructions

SIMD: 1 instruction, 8 results

a        

+

b        

c        

one 512-bit register = 8 × 64-bit lanes
(or 16 × 32-bit, or 64 × 8-bit)

The instruction sets

- **x64**: SSE2 (baseline, 128-bit), AVX2 (256-bit), AVX-512 (512-bit)
- **ARM**: NEON (128-bit, baseline on ARMv8), SVE/SVE2 (scalable)
- **RISC-V**: RVV (scalable)
- **POWER**: AltiVec/VSX · **LoongArch**: LSX, LASX
- **WebAssembly**: 128-bit SIMD, in every browser

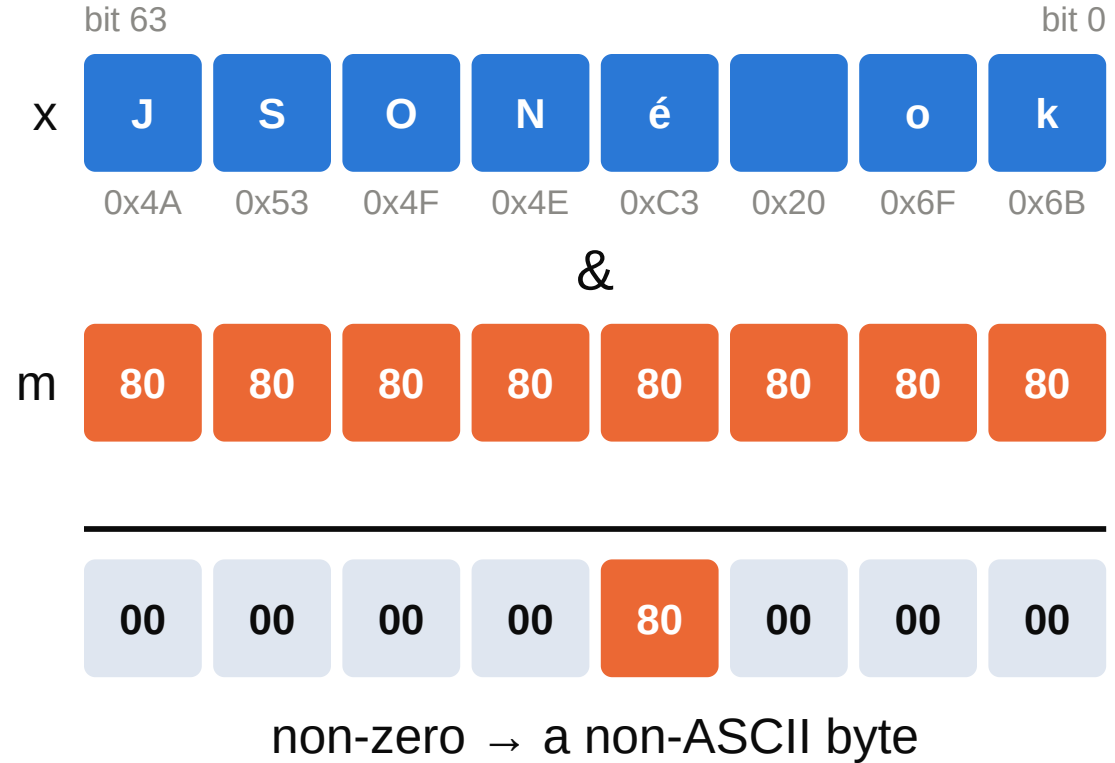
Portability: compile several kernels, dispatch at runtime on CPU features. C++26 adds data-parallel types (`std::simd`).

SWAR: SIMD within a register

- Use plain 64-bit integer instructions
- A 64-bit register holds 8 bytes
- Fully portable C, C++, Rust, Go, Java...
- No intrinsics, no dispatch, no `#ifdef`
- Requires some cleverness

A great place to start.

Is every byte ASCII?



```
m = 0x8080808080808080;  
ascii = ((x & m) == 0);
```

8 bytes checked with one 64-bit AND

Check whether we have 8 digits

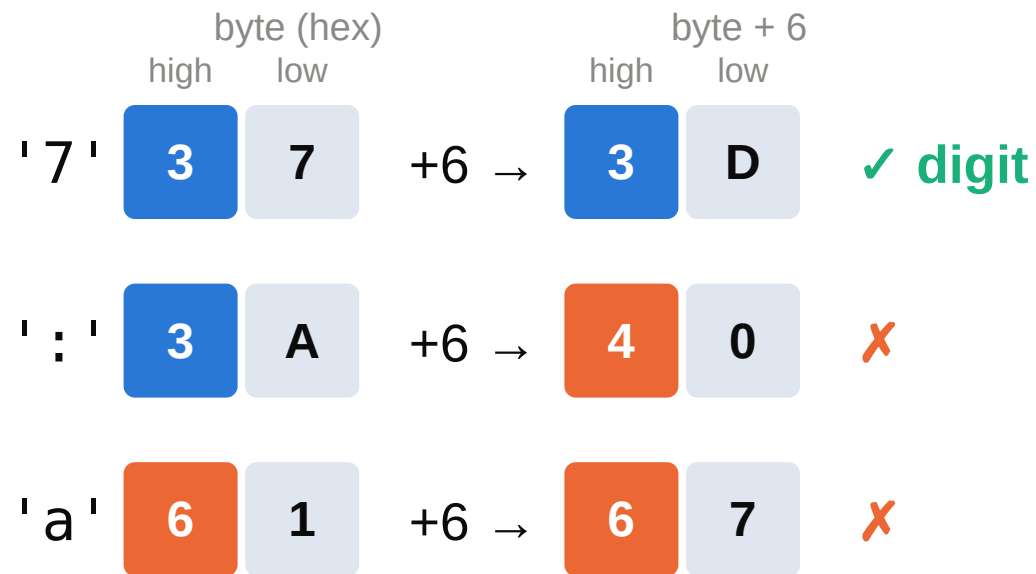
In ASCII/UTF-8, the digits 0, 1, ..., 9 have values

0x30, 0x31, ..., 0x39.

To recognize a digit:

- The high nibble should be 3.
- The high nibble should remain 3 if we add 6 (0x39 + 0x6 is 0x3f)

Is this byte a digit?



Digit ⇔ high nibble is 3 before *and* after +6

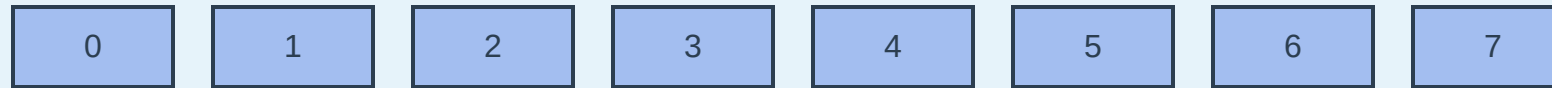
0x30 ... 0x39 → 0x36 ... 0x3F

0x3A ... 0x3F → 0x40 ... 0x45 (carries out)

Same trick on 8 bytes at once: next slide

ASCII processing flow

ASCII numbers



ASCII codes (hex)



Zero lower 4 bits (AND 0xF0)



Add 6, shift right 4



OR result (hex)



```
// load 8 input bytes into val
bool is_made_of_eight_digits_fast(uint64_t val) noexcept {
    return !(((val + 0x4646464646464646)
              | (val - 0x3030303030303030))
            & 0x8080808080808080));
}
```

compiles to

```
add    rax, rdi
add    rdi, rdx
or     rax, rdi
test   rax, rdx
```

Four instructions for eight characters, and no branch.

Case study: ASCII processing

ASCII to lower case

```
For each character c
  If c - 'A' <= 'Z' - 'A' then
    c = c + 'a' - 'A'
  EndIf
EndFor
```

One byte per iteration. One unpredictable branch per byte.

64 characters in 3 instructions

input c

`A = set1('A'), Z_A = set1(25), a_A = set1(32)`



1. subtract 'A'

`ca = _mm512_sub_epi8(c, A)`

wraps around when $c < 'A'$



2. compare ≤ 25 , unsigned

`mask = _mm512_cmple_epi8_mask(ca, Z_A)`

one compare checks both bounds



3. add 32 where mask is set

`out = _mm512_mask_add_epi8(c, mask, c, a_A)`

other lanes keep c



3 instructions, 64 characters, no branch

Let us check with llvm-mca

- `llvm-mca` is a static machine-code analyzer shipped with LLVM
- Feed it assembly, name a microarchitecture, get a cycle estimate
- No hardware required, no noise, no warm-up

```
llvm-mca -mcpu=icelake-server -iterations=100 kernel.s
```

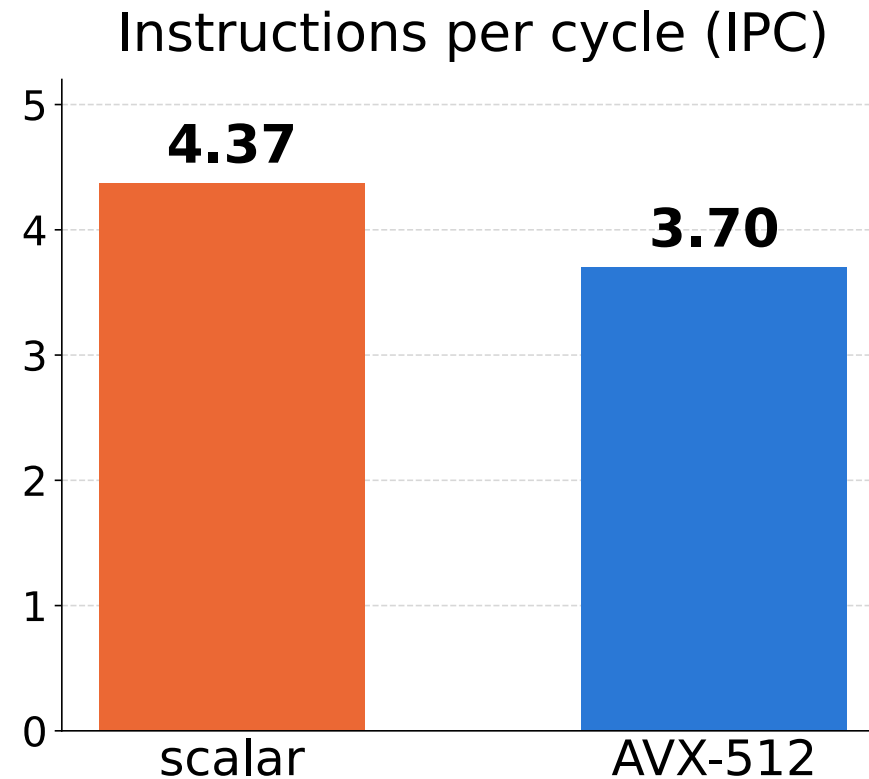
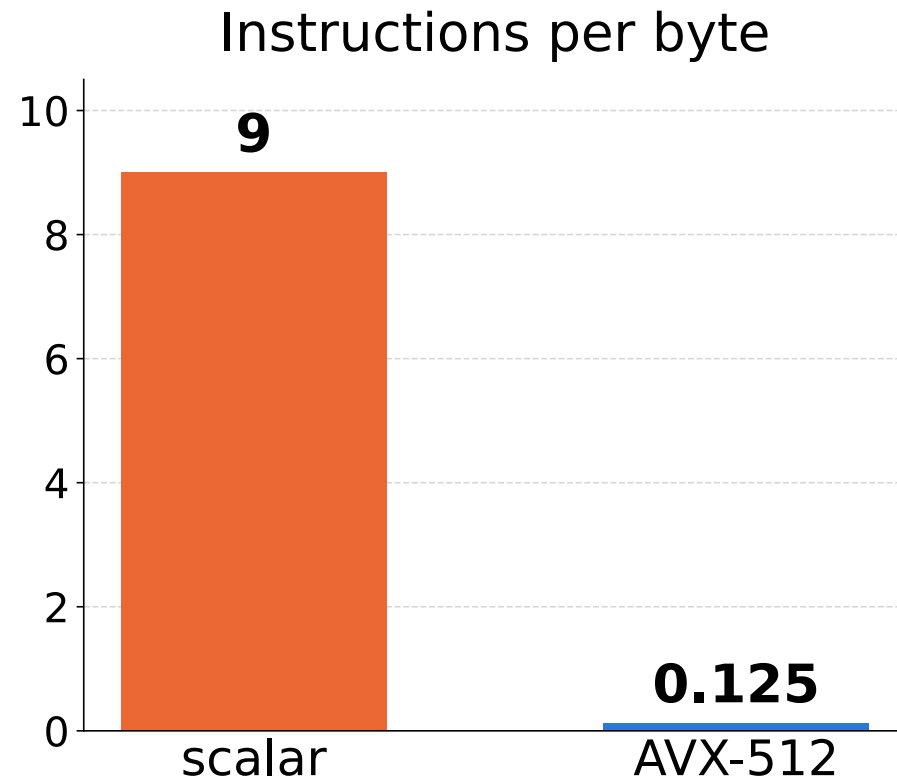
The scalar loop: 1 byte per iteration

```
movzbl    (%rdi,%rax), %ecx
leal      -65(%rcx), %edx
cmpb      $26, %dl
jae       .LBB1_2
addb      $32, %cl
movb      %cl, (%rdi,%rax)
addq      $1, %rax
cmpq      %rax, %rsi
jne       .LBB1_1
```

The AVX-512 loop: 64 bytes per iteration

```
vmovdqu64    (%rdi,%rax), %zmm2
vpsubb       %zmm0, %zmm2, %zmm3
vpcmpub      $2, %zmm1, %zmm3, %k1
vpaddb       %zmm4, %zmm2, %zmm2 {%k1}
vmovdqu64    %zmm2, (%rdi,%rax)
addq         $64, %rax
cmpq         %rax, %rsi
jne          .LBB0_1
```

llvm-mca says...



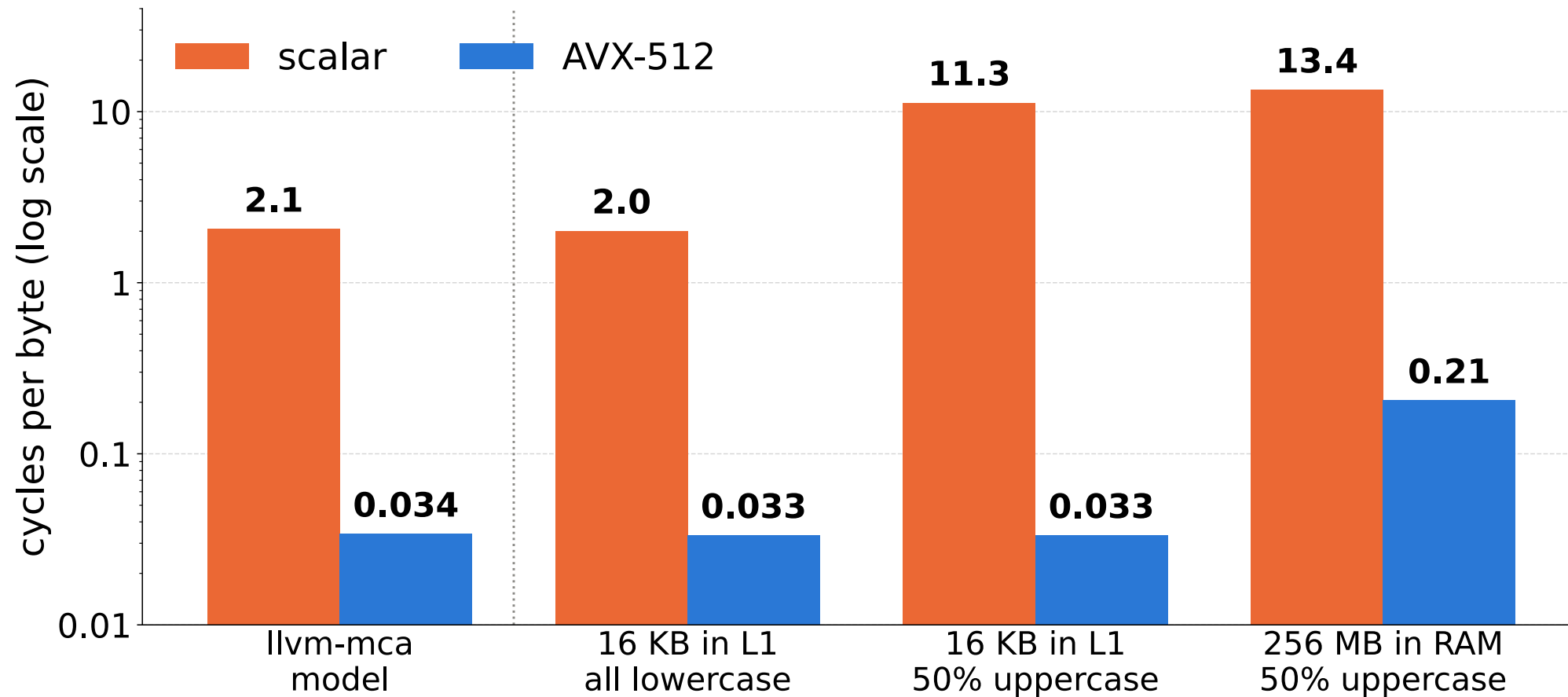
About 60× fewer cycles per byte.

What llvm-mca will not tell you

- It assumes perfect branch prediction
- It assumes every load hits L1
- It often ignores memory alignment issues
- It models one microarchitecture at a time.

It is a model. Not reality.

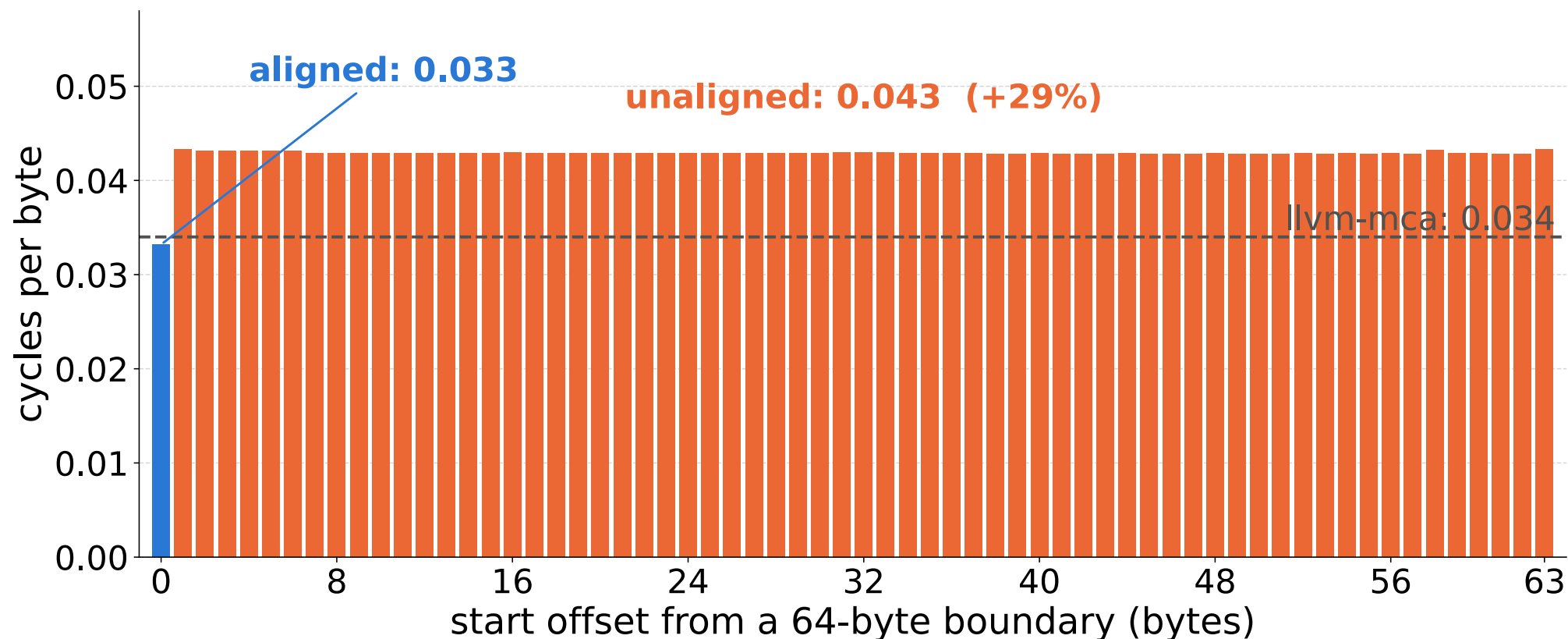
Reality: Intel Xeon Gold 6548N (Emerald Rapids)



Model vs reality

- Model assumptions hold (L1, predictable, aligned): **within 3%** of llvm-mca
- Unpredictable branches: scalar is **5× slower** than the model
- 256 MB in RAM: AVX-512 is **6× slower** than the model (memory-bound, ~17 GB/s)
- Reality: AVX-512 is **~65× faster** than scalar on real text

Alignment: 64-byte loads and stores, 64 offsets



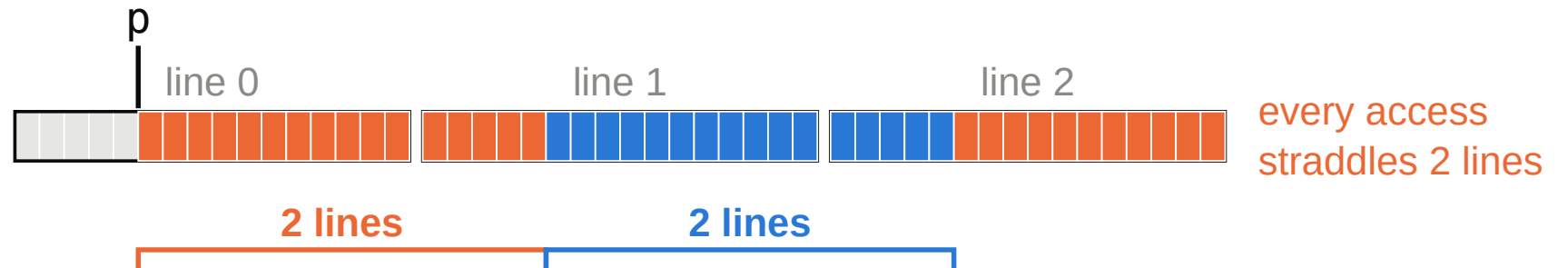
Only 1 offset in 64 matches the model. The other 63 straddle two cache lines: **+30%**.

Masked loads and stores

- AVX-512 loads and stores take a 64-bit mask: one bit per byte
- Masked-out bytes are not read, not written, and **cannot fault**
- Round `p` down to the cache line, mask off the bytes before `p`: the rest of the loop is aligned
- Same trick handles the tail: no scalar loop, no branch

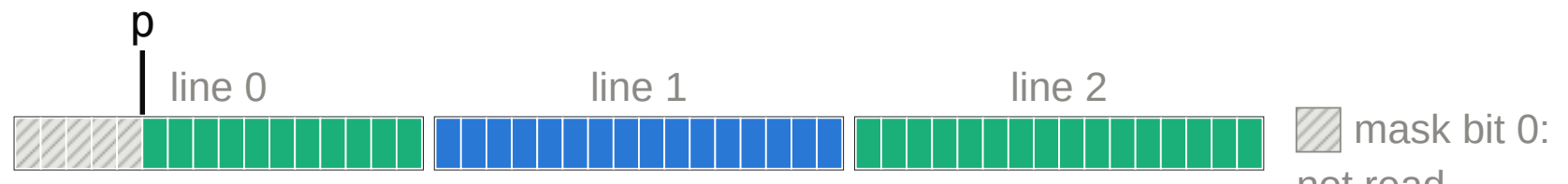
Naive

64-byte loads at `p`



Masked head

$p_0 = p \& \sim 63$



Case study: JSON

JSON

- Portable, simple, human-readable; used by ~97% of API requests
- Strings (escaped), numbers, objects, arrays
- Reading and writing JSON is often *slow*: 100 MB/s to 300 MB/s
- Slower than a fast disk or a fast network

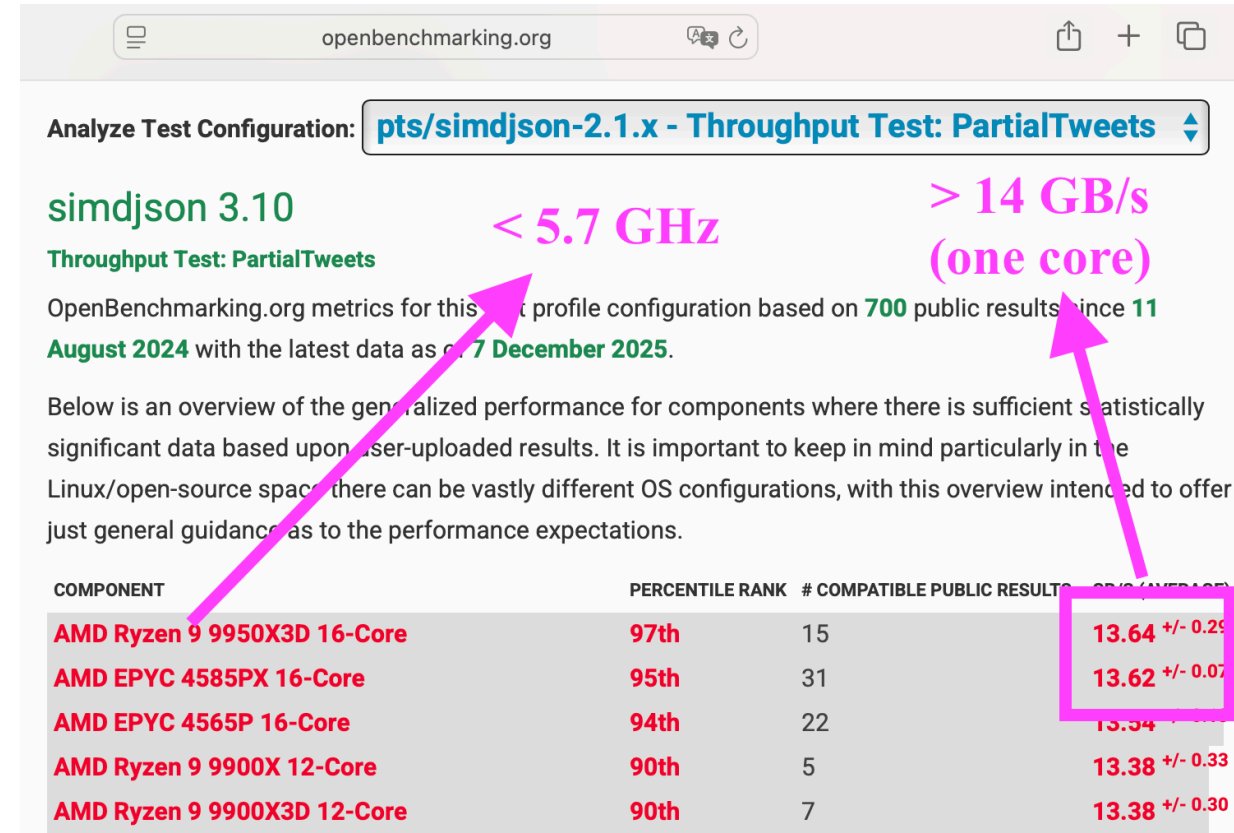
```
$ go run parse_twitter.go  
Parsed 0.63 GB in 6.961 seconds (90.72 MB/s)
```

"But JSON parsing is inherently serial"

- Every byte can change the meaning of every following byte.
- Strings contain escapes; escapes contain quotes.
- It is a textbook state machine.

This was the conventional wisdom. It was wrong.

- openbenchmarking.org
- 14 GB/s, at less than 5.7 GHz
- Parsing JSON at better than **2.5 bytes per cycle**



You are probably using simdjson

- Node.js, Bun, Deno, Electron
- ClickHouse
- WatermelonDB, Apache Doris, Meta Velox, Milvus, QuestDB, StarRocks



```
{  
  "nom": "Marie Dubois",  
  "age": 28,  
  "ville": "Paris",  
  "loisirs": ["lecture", "voyage", "yoga"],  
  "contact": {  
    "email": "marie.dubois@example.com",  
    "phone": "+33123456789"  
  }  
}
```

Every major JavaScript engine parses JSON with SIMD

- `JSON.parse` in Node.js, Bun and Deno is data-parallel.
- Billions of calls per second worldwide.
- Nobody had to change a single line of JavaScript.

simdjson: two-stage design

Stage 1 (data-parallel): scan the whole document with SIMD

- find every structural character and the start of every string
- validate UTF-8
- produce an index

Stage 2 (mostly serial): walk the index and build values

- the hard, branchy work now runs on ~5% of the bytes

simdjson: the numbers

- Structural scan: about **10 GB/s**
- UTF-8 validation: about **30 GB/s**
- Minification: **10 to 20 GB/s**
- Fast skipping: only parse what you actually read

Classifying characters

We need to sort every byte into a class:

- comma (0x2c) ,
- colon (0x3a) :
- brackets (0x5b, 0x5d, 0x7b, 0x7d): [,], {, }
- white-space (0x09, 0x0a, 0x0d, 0x20)
- everything else

A switch statement per byte? No.

Vectorized classification

- Most SIMD instruction sets support 'vectorized lookup tables' (at least 16-element)
- With a 256-element table we could just compute `H(c)`
- With 16-element tables, we need two tables `H1` and `H2`
- Find `H1` and `H2` such that the bitwise AND of the lookups classifies the character:

```
H1(c & 0xf) & H2(c >> 4)
```

```
low_nibble_mask  = {16, 0, 0, 0, 0, 0, 0, 0, 0, 8, 12, 1, 2, 9, 0, 0};  
high_nibble_mask = {8, 0, 18, 4, 0, 1, 0, 1, 0, 0, 0, 3, 2, 1, 0, 0};
```

Five instructions, 16 to 64 bytes at a time:

```
nib_lo  = input & 0xf;  
nib_hi  = input >> 4;  
shuf_lo = lookup(low_nibble_mask, nib_lo);  
shuf_hi = lookup(high_nibble_mask, nib_hi);  
return shuf_lo & shuf_hi;
```

This trick generalizes: any 256-way classification into 8 classes.

low nibble (c & 0xF) → low_nibble_mask

high nibble (c >> 4) → high_nibble_mask

		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
		16	0	0	0	0	0	0	0	0	8	12	1	2	9	0	0
0	8										\t	\n			\r		
1	0																
2	18	␣	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	4	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	0	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	1	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	0	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	1	p	q	r	s	t	u	v	w	x	y	z	{		}	~	
8	0																
9	0																
A	0																
B	3												⚡	⚡	⚡		
C	2													⚡			
D	1												⚡		⚡		
E	0																
F	0																

class = bit

 1 [] {}

 2 , comma

 4 : colon

 8 \t \n \r

 16 space

 0 everything else

 ≥ 0x80: never structural

row value & column value

= class of the byte

e.g. '{' = 0x7b:

high[7] & low[b] = 1 & 1 = **1**

Classification, 64 bytes at a time

input

64 bytes

, : { _ \n a " 5] \r

high nibble

input >> 4

2 3 7 2 0 6 2 3 5 0

low nibble

input & 0xF

C A B 0 A 1 2 5 D D

↓ two 16-entry table lookups ↓

H[high]

lookup(high_nibble_mask, .)

18 4 1 18 8 0 18 4 1 8

L[low]

lookup(low_nibble_mask, .)

2 12 1 16 12 0 0 0 9 9

&

class

H[high] & L[low]

2 4 1 16 8 0 0 0 1 8

5 instructions per 64 bytes; 1 = bracket, 2 = comma, 4 = colon, 8 = `\t\n\r`, 16 = space, 0 = other

When the instruction set gives you the instruction: SVE2

- ARM SVE2 has `match` : input vector, a 16-byte set, one predicate bit per byte in the set
- NEON has nothing like it; on x64 the closest thing (`pcmpqstrm`) is slow
- Our structural-character classifier: **4 NEON instructions** → **1**
- The catch: a predicate lives in a predicate register; materialize it with `svsel` + weights, reduce with `addp`
- NEON–SVE bridge (`svset_neonq_u8` , `svget_neonq_u8`): mix both, no assembly

NEON vs SVE2 match, one 16-byte block

16 input bytes

`d0`

{ " i d " : [7 , { " k " : 2 }

NEON

1–2. add 3, high nibble

`vshrq_n_u8(vaddq_u8(d0,3),4)`

7 2 6 6 2 3 5 3 2 7 2 6 2 3 3 8

3. table lookup

`vqtbl1q_u8(op_table, nibble)`

{ ,]] , : [: , { ,] , : : }

4. compare with input

`vceqq_u8(lookup, d0)`

ff 00 00 00 00 ff ff 00 ff ff 00 00 00 ff 00 ff

SVE2

1. match against a set

`svmatch_u8(pg, d0, operators)`

1 0 0 0 0 1 1 0 1 1 0 0 0 1 0 1 predicate

→ mask

predicate → bytes

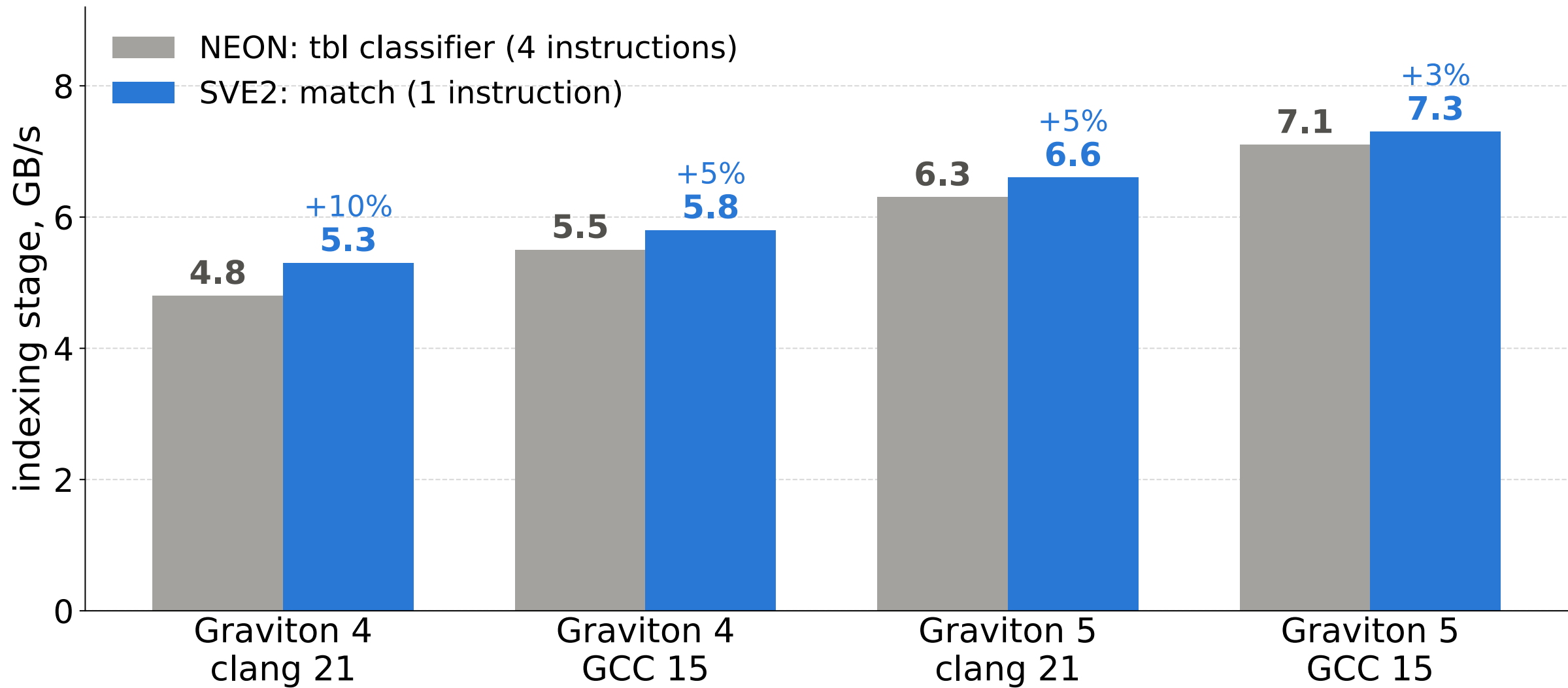
`svsel_u8(p, weights, 0)`

`3 × vpaddq_u8` over four blocks → 64-bit mask

01 00 00 00 00 20 40 00 01 02 00 00 00 20 00 80

= 0xA361... : bit i set ⇔ byte i is structural

simdjson indexing stage, AWS Graviton 4 and 5



Whole parse: +2–4% on Graviton 4, +1–2% on Graviton 5. Madhurendra Purbay (ARM), simdjson PR 2866, 51

Serialization is also a data-parallel problem

- JSON requires escaping `"`, `\`, and control characters.
- Almost no string actually needs escaping.
- So: check 16 to 64 bytes at once, and take the fast path.

SIMD string escaping

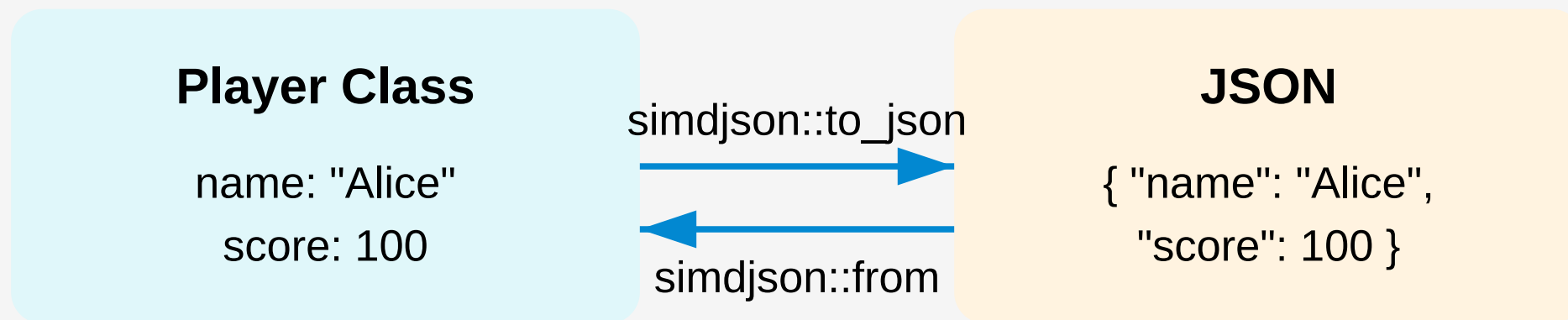
Traditional (1 byte at a time):

```
for (char c : str) {  
    if (c == '"' || c == '\\\' || c < 0x20)  
        return true;  
}
```

SIMD (64 bytes at once):

```
auto chunk = load_64_bytes(str);  
auto needs_escape = check_all_conditions_parallel(chunk);  
if (!needs_escape)  
    return false; // Fast path!
```

C++26 compile-time reflection

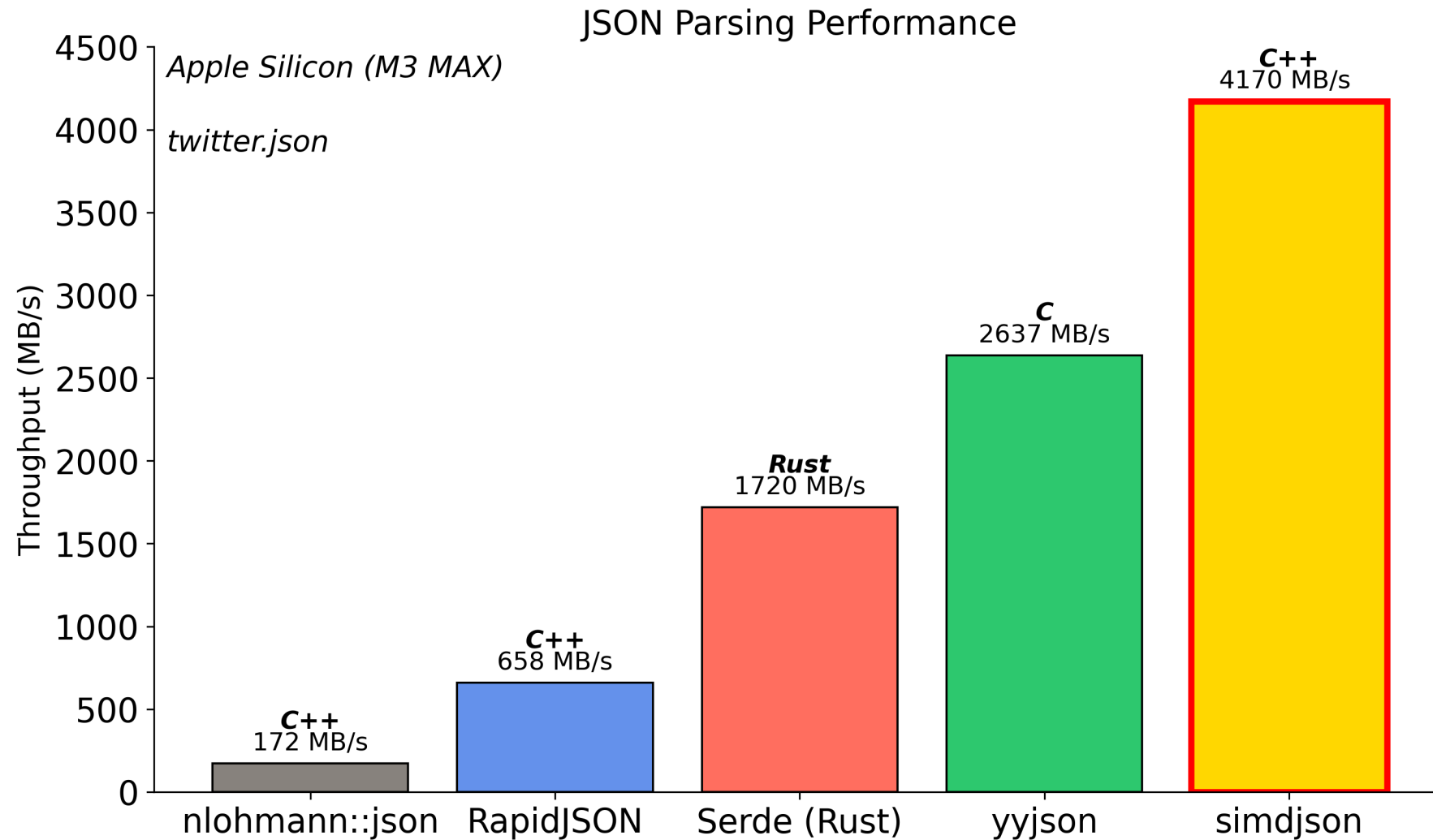


One line each way (C++26)

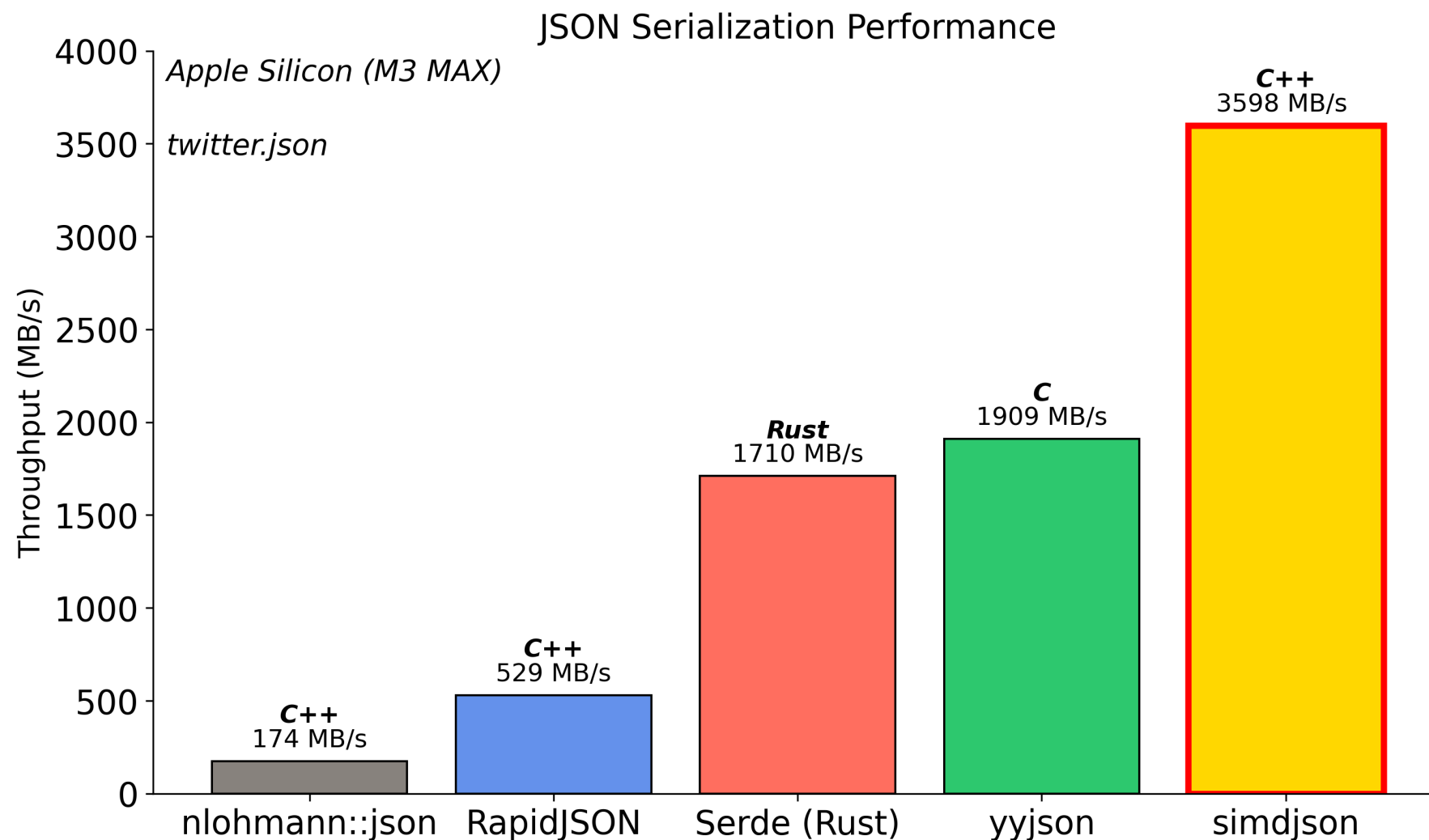
```
struct Player {  
    std::string username;  
    int level;  
};  
  
Player load_player(std::string& json_str) {  
    return simdjson::from(json_str);  
}  
  
std::string save_player(const Player& p) {  
    return simdjson::to_json(p);  
}
```

No macros. No code generation step. No runtime reflection cost.

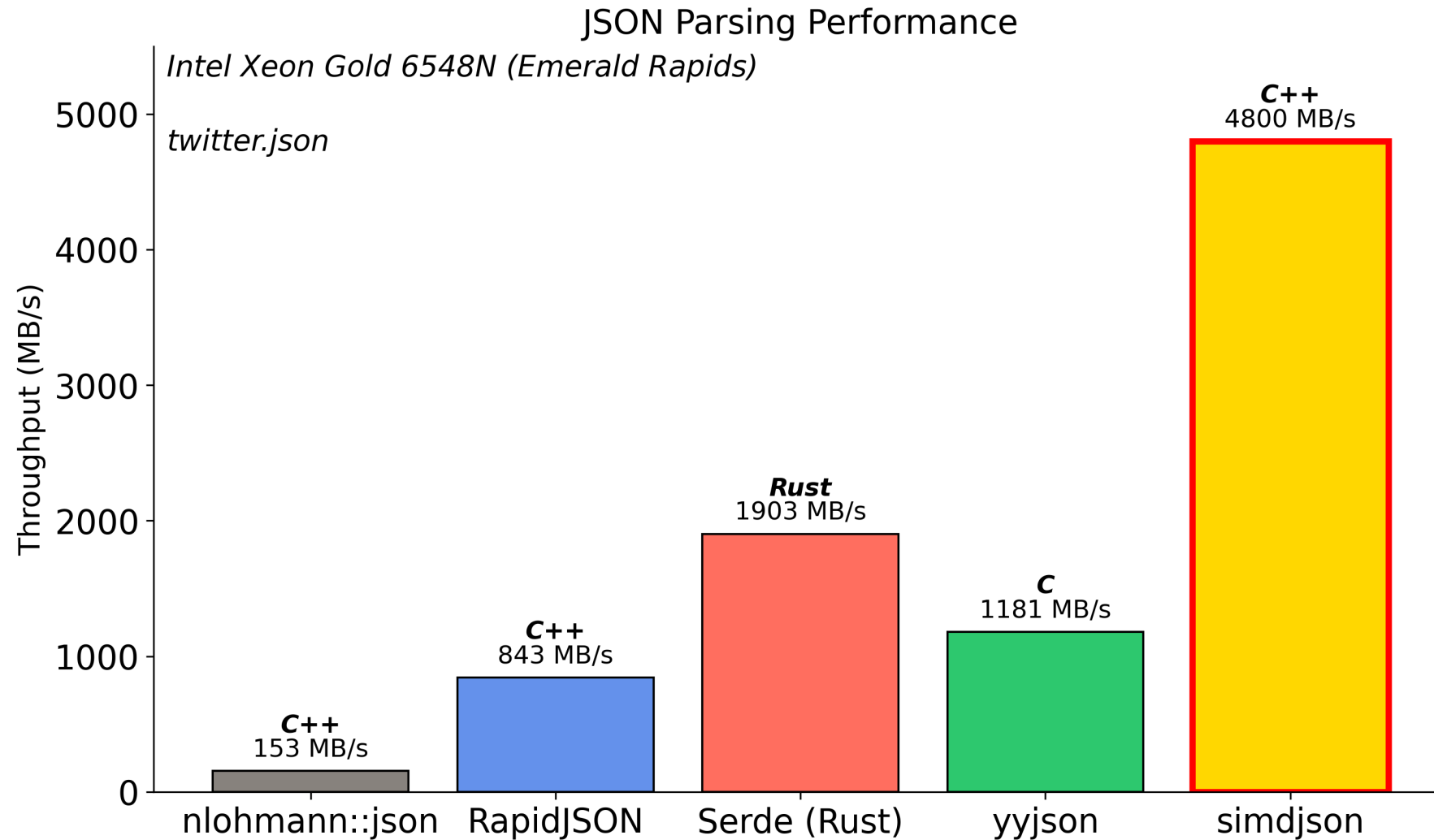
Deserialization (Apple Silicon)



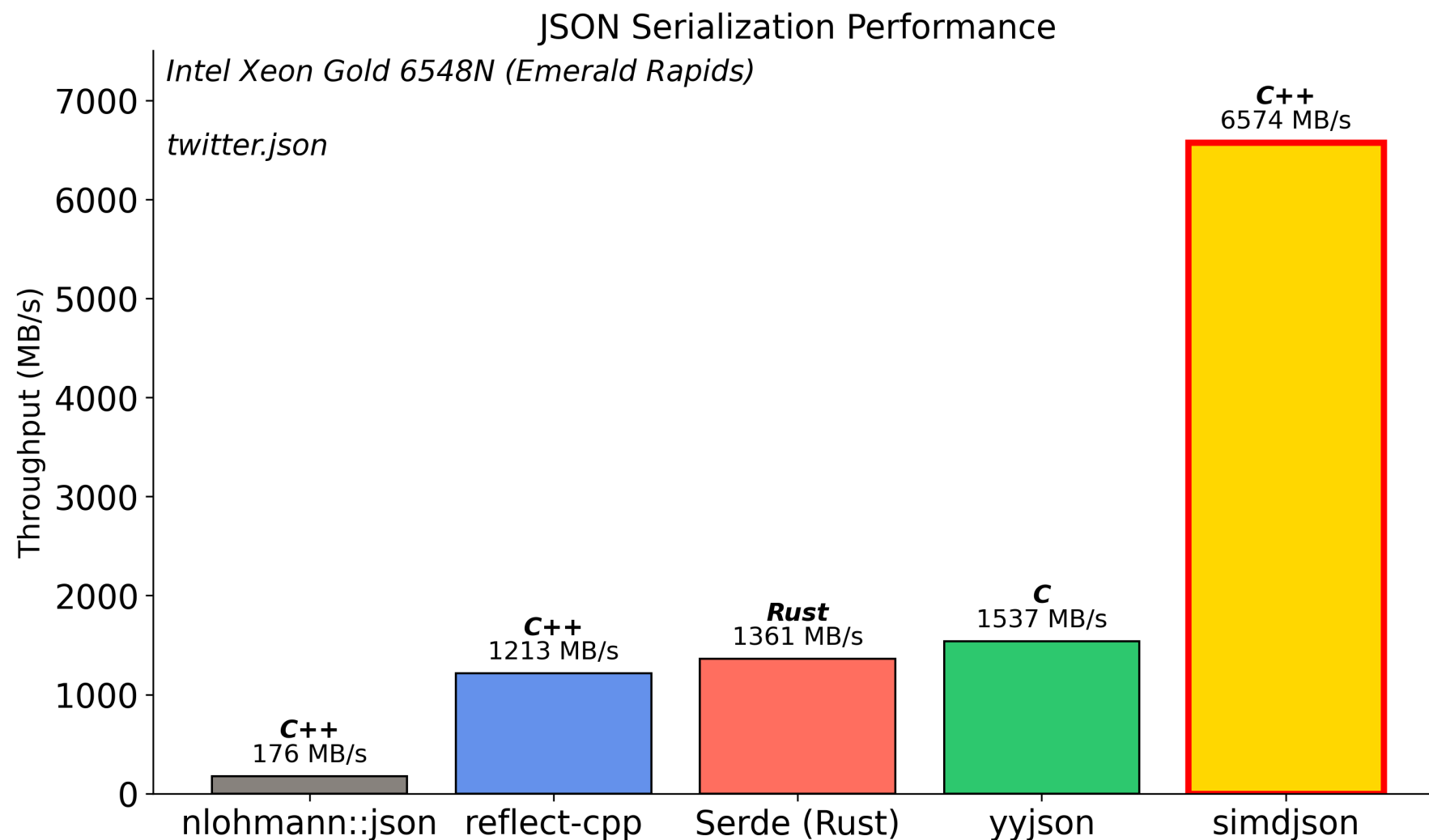
Serialization (Apple Silicon)



Deserialization (Intel Xeon Gold 6548N)



Serialization (Intel Xeon Gold 6548N)



Case study: IP addresses

192.168.0.1 → **32 bits**

- Every server logs the client address. Every firewall evaluates a rule. Every URL parser meets an IP literal.
- The usual tool: `inet_pton` from the C library, ~340 instructions per address.
- An IPv4 address is at most 15 bytes: it fits in **one** 16-byte register.
- An IPv6 address is at most 45 bytes: it fits in **one** 64-byte register.

Load the whole address once. Never walk it.

IPv4 in six steps, no loop

1. masked load, len = 11

`mask_loadu_epi8(pad='0', len)`
pad with '0': neither dot nor error

2. dots → partition p

`p = cmpeq(v, '.') | 1 << len`
`p = 0xA88`: the whole layout in 16 bits

3. perfect hash → key

`k = (p * 0x00CF7800) >> 24`
81 layouts → 81 distinct keys

4. table shuffle

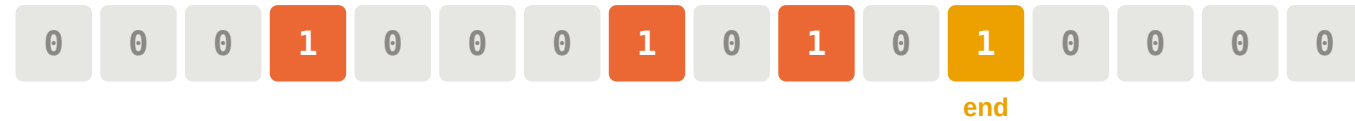
`shuffle_epi8(v, table[k])`
octet → [0, hundreds, tens, ones]

5. VNNI dot product

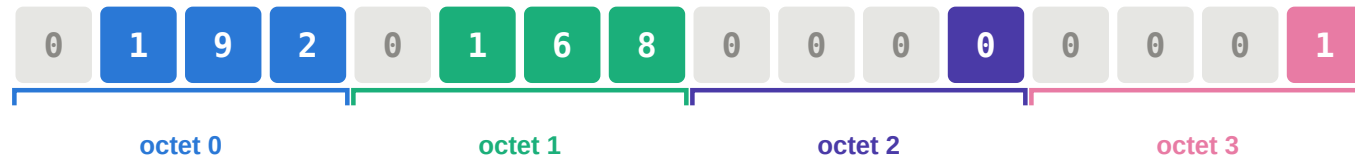
`dpbusd_epi32(digits, weights)`
weights [0, 100, 10, 1] × 4

6. validate

`p == aux[k] | non-digit | > 255`
all in mask registers: one branch



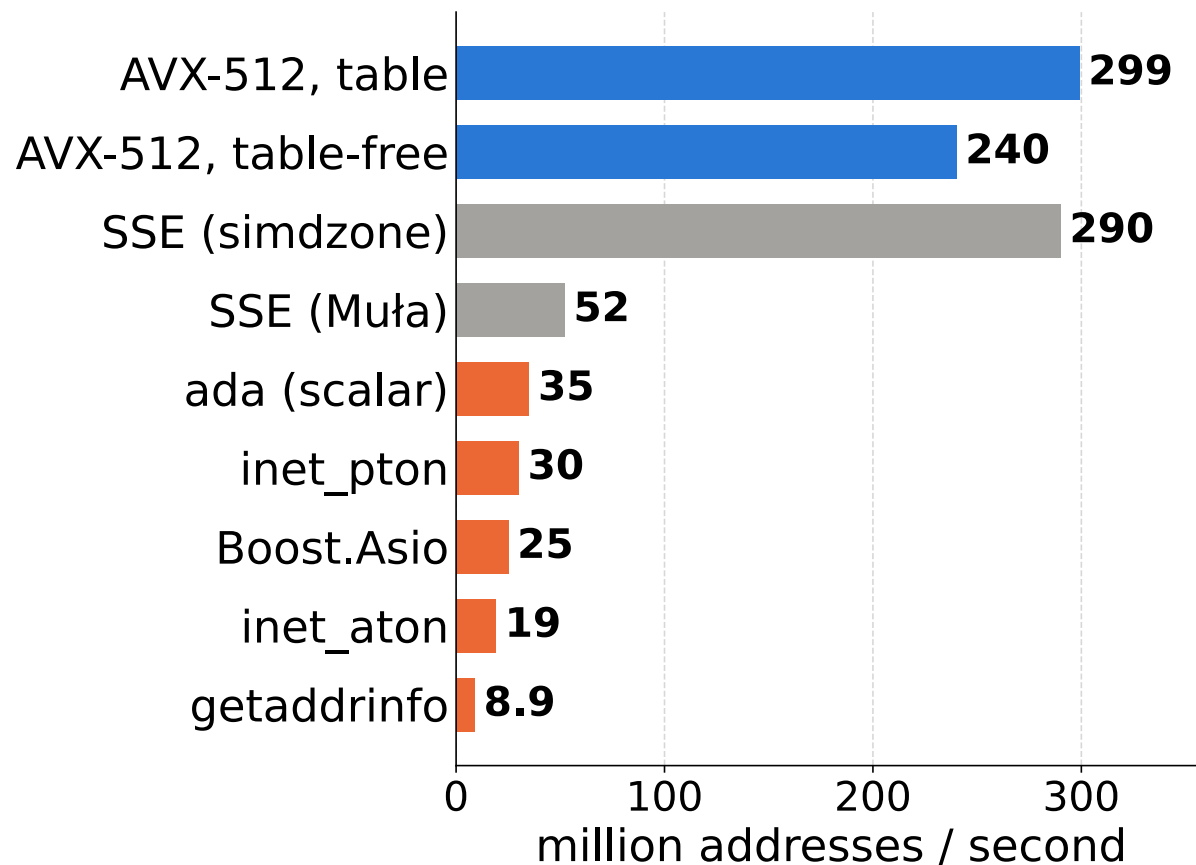
$0xA88 \times 0x00CF7800 \rightarrow k = 136 \rightarrow \text{shuffle_table}[136], \text{aux_table}[136]$



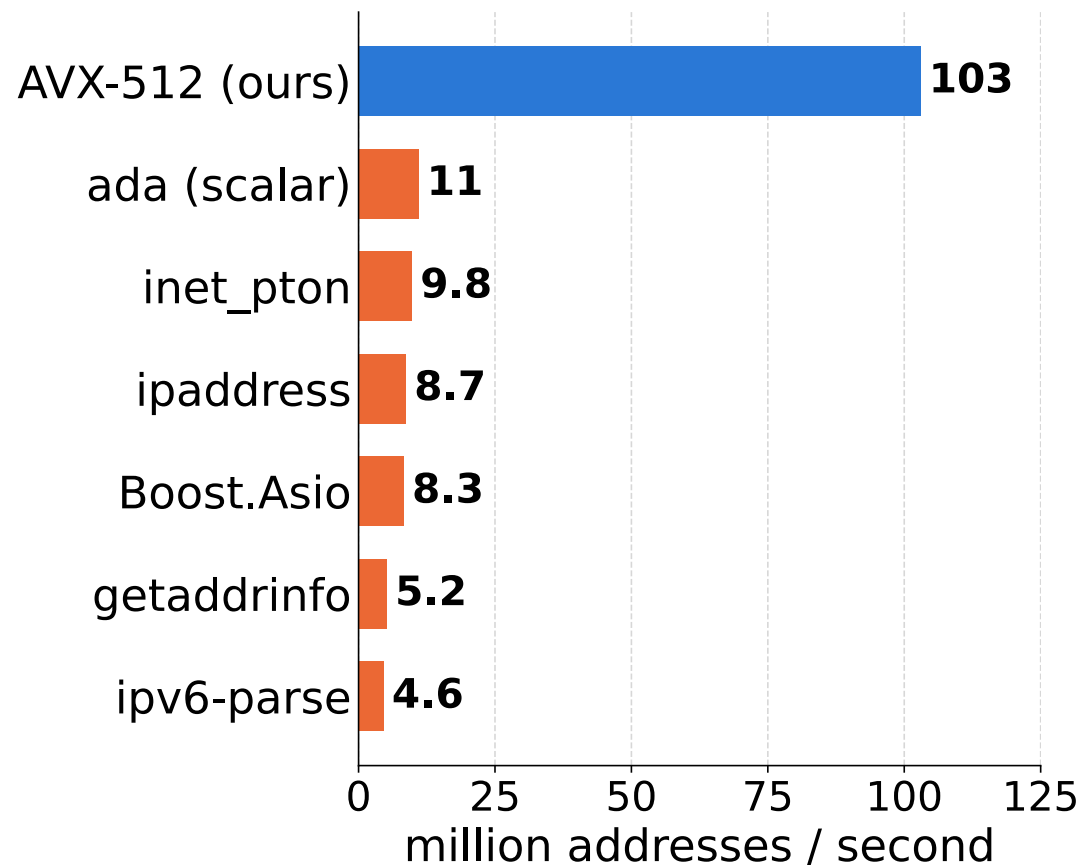
$192.168.0.1 \rightarrow 0xC0A80001 \cdot \sim 50 \text{ instructions, no loop, no per-digit branch}$

IPv4 parsing, Intel Xeon Gold 6548N

IPv4, random addresses

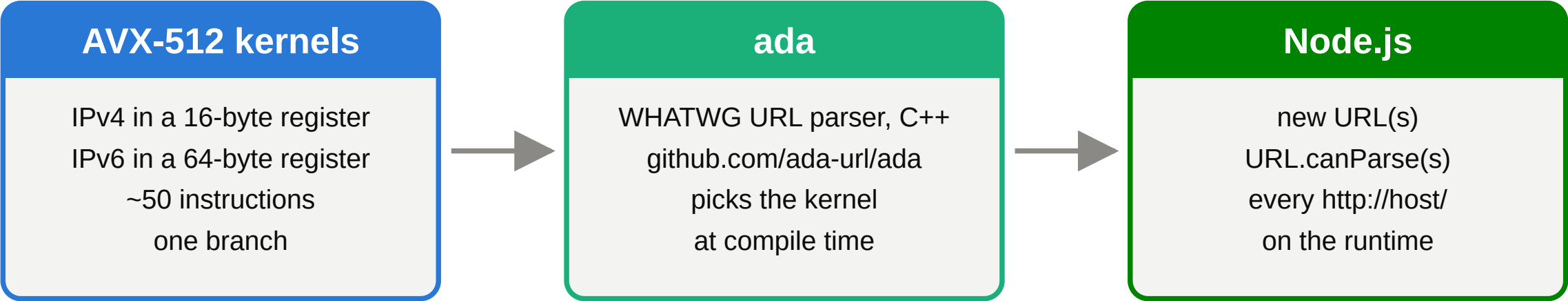


IPv6, traffic-like addresses



10× over `inet_pton` on IPv4, 10× on traffic-like IPv6. 6–60× fewer instructions.

Deployed: ada → Node.js



Measured in Node.js, Xeon Gold 6548N, 100 000 URLs per corpus, best of 5:

	new URL (s)	URL . canParse (s)	
IPv4 hosts	+8%	+13%	2 300–3 200 instructions per new URL; the address parser was a few hundred of them
IPv6, random	+8%	+20%	
IPv6, traffic-like	+2%	+9%	

Case study: perfect hashing

The lookup *is* the workload

- Map a string to a value: HTTP method, header name, URL scheme, keyword, MIME type
- The keys are **fixed when you write the code**
- Yet we hash, mask, probe, chase a pointer, compare: `std::unordered_map`, 12.8 ns
- With the keys known at compile time, we can build a **perfect hash**: no collisions, one candidate per slot

Library: github.com/ConstexprCore/perfect_hash (with Francisco Geiman Thiesen)

One hash, one comparison

$$\text{slot} = (\text{len} + \text{asso}[\text{key}[0]]) \& 7$$

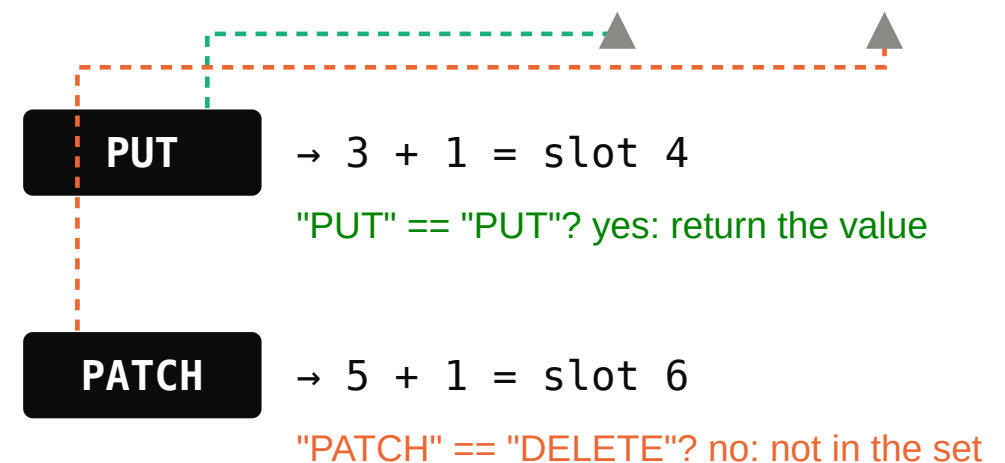
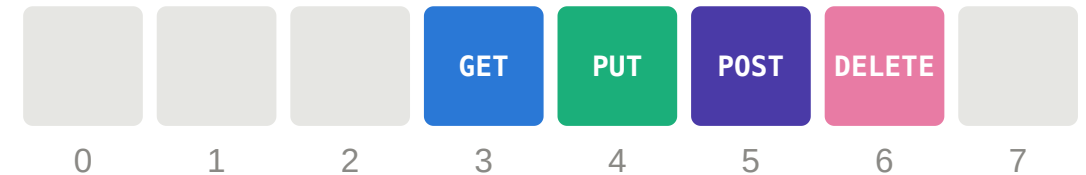
keys, known at compile time

key	len	key[0]	asso	slot
GET	3	G	0	3
PUT	3	P	1	4
POST	4	P	1	5
DELETE	6	D	0	6

asso: G → 0, P → 1, D → 0

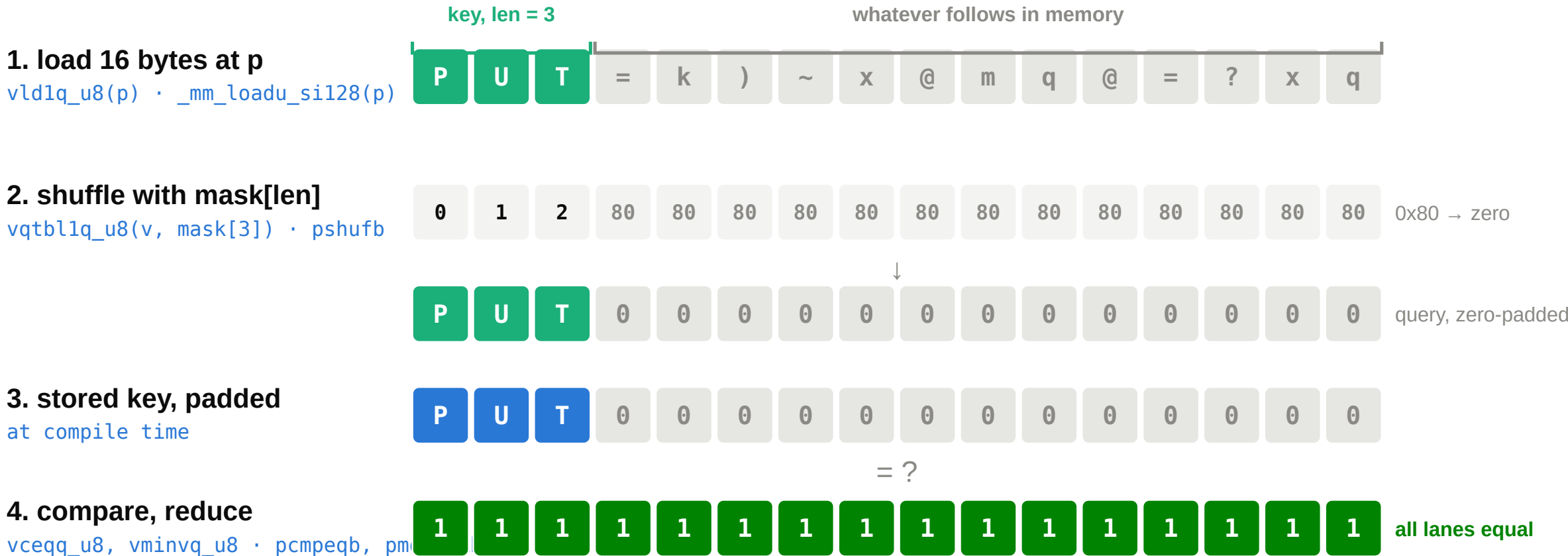
found by a bounded greedy search
at compile time (constexpr)

8 slots, one candidate each



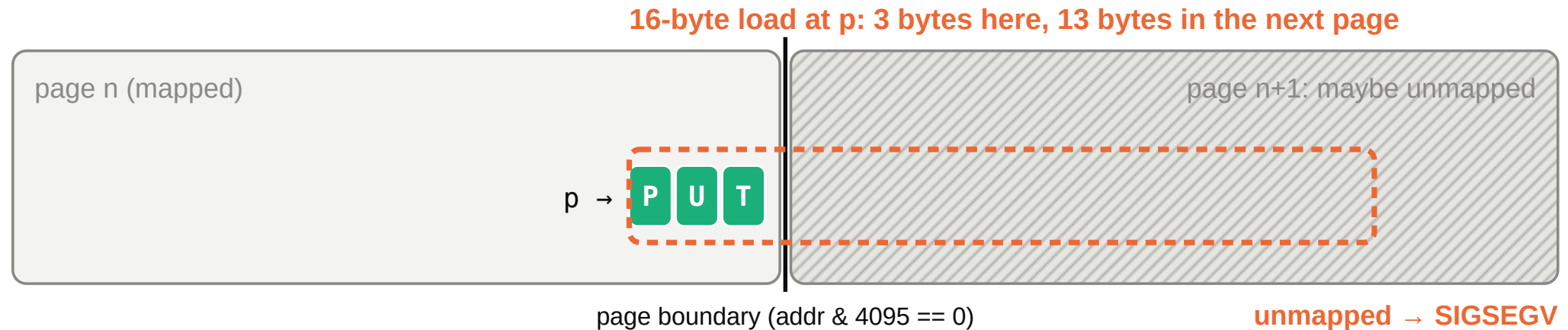
one hash, one comparison, no probing, no chain

Compare the whole key at once



The memory page trick

memory, in 4 KB pages



a load that stays inside one page cannot fault

the OS maps memory page by page; the protection unit is 4 KB, never a single byte

```
if ((addr & 4095) <= 4096 - 16) [[likely]]
    return load16(p); // wide, branch-free
copy len bytes to a 16-byte buffer; // rare
return load16(buffer);
```

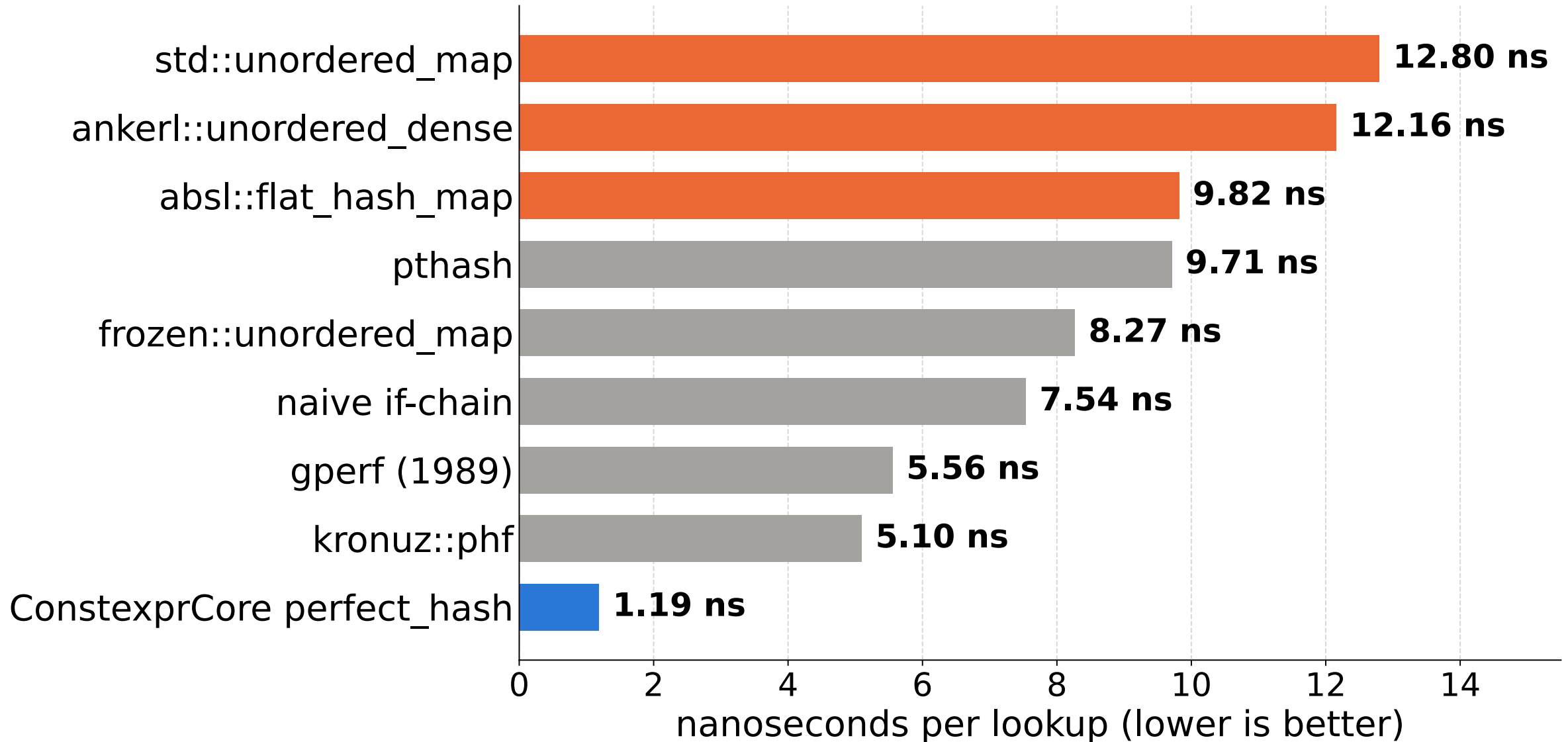
one branch, taken 99.6% of the time

(15 of every 4096 addresses)

the predictor learns it instantly;

8 unpredictable branches → 1 predictable one

URL schemes, 6 keys, Apple M3 Max



1.19 ns, 4.9 cycles, 8.7 instructions per cycle, **zero** branch mispredictions.

Case study: Unicode

simdutf

- Inside Safari, Chrome, Node.js, Bun
- Unicode transcoding and validation at gigabytes per second
- Base64 too
- x64, ARM, POWER, RISC-V, LoongArch



simdutf

Unicode (UTF-16)

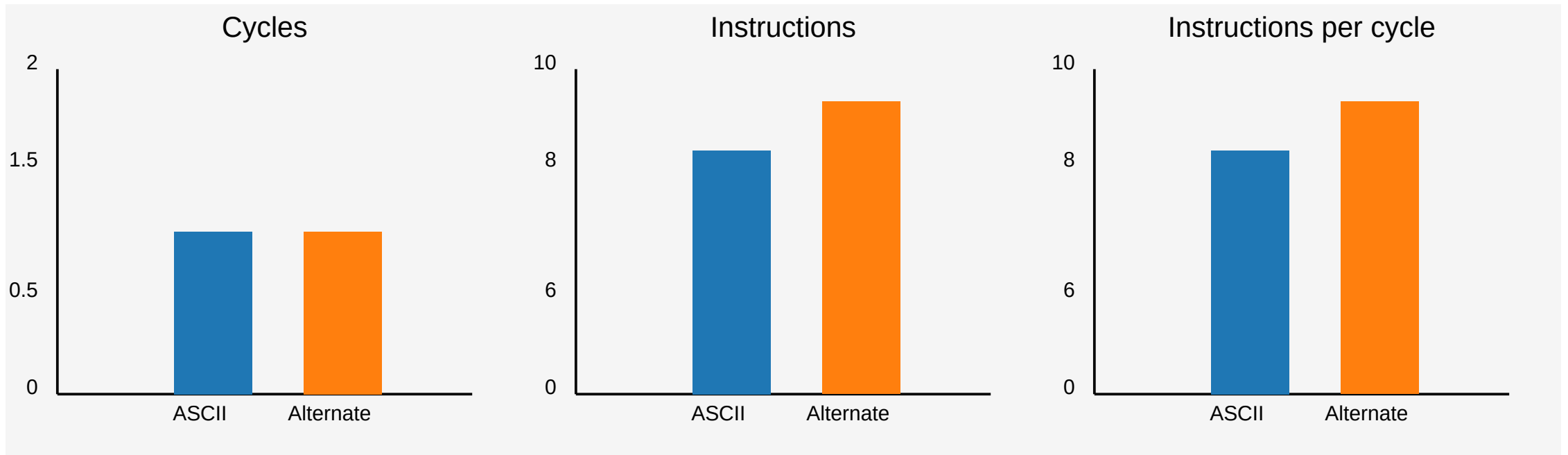
- Code points from U+0000 to U+FFFF: a single 16-bit value.
- Beyond: a *surrogate pair*, U+D800 – U+DBFF followed by U+DC00 – U+DFFF .
- A lone surrogate makes the string ill-formed.

Every JavaScript string, every Java string, every Windows filename.

Validate

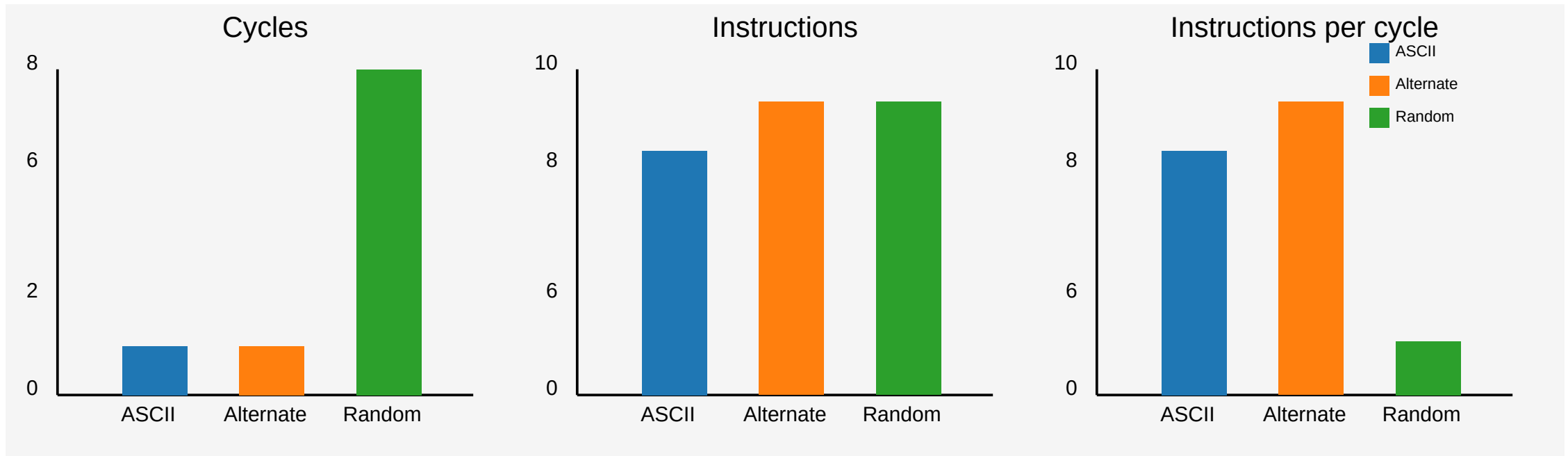
```
PROCEDURE validate_utf16(code_units)
  i ← 0
  WHILE i < |code_units|
    unit ← code_units[i]
    IF unit ≤ 0xD7FF OR unit ≥ 0xE000 THEN
      INCREMENT i
      CONTINUE
    IF unit ≥ 0xD800 AND unit ≤ 0xDBFF THEN
      IF i + 1 ≥ |code_units| THEN
        RETURN false
      next_unit ← code_units[i + 1]
      IF next_unit < 0xDC00 OR next_unit > 0xDFFF THEN
        RETURN false
      i ← i + 2 // Valid surrogate pair
      CONTINUE
    RETURN false
  RETURN true
```

Performance results (Apple M4)



1 character per cycle might be just 4 GB/s — slower than your disk.

Now make the input adversarial



We are now barely at 1 GB/s. **The branch predictor was doing the work.**

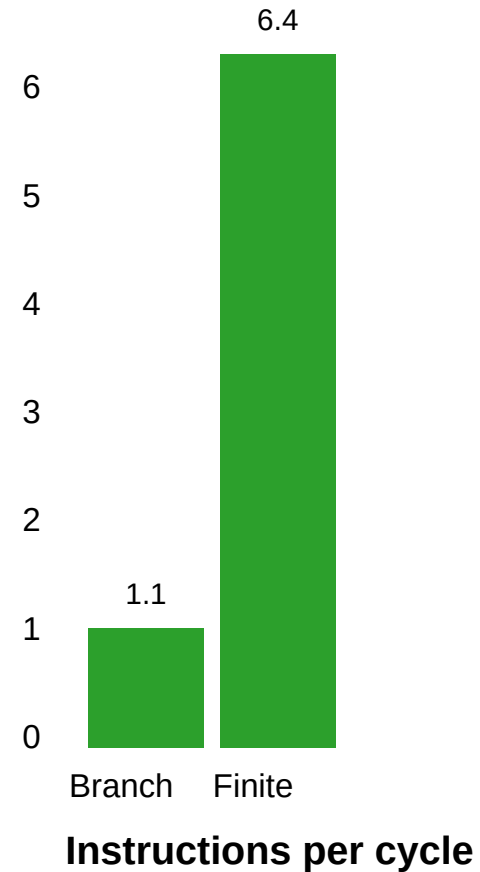
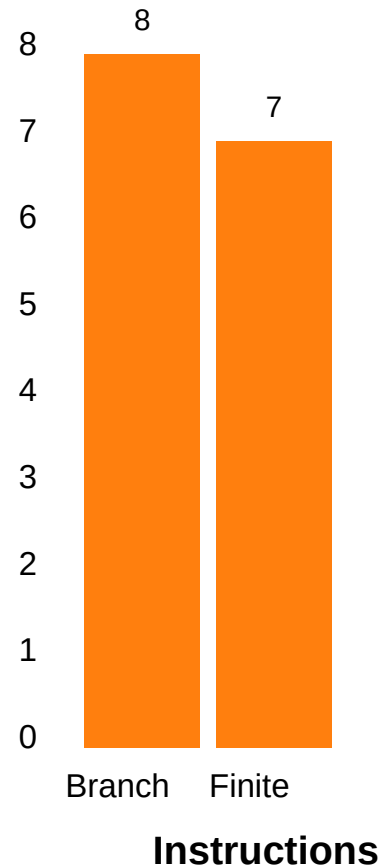
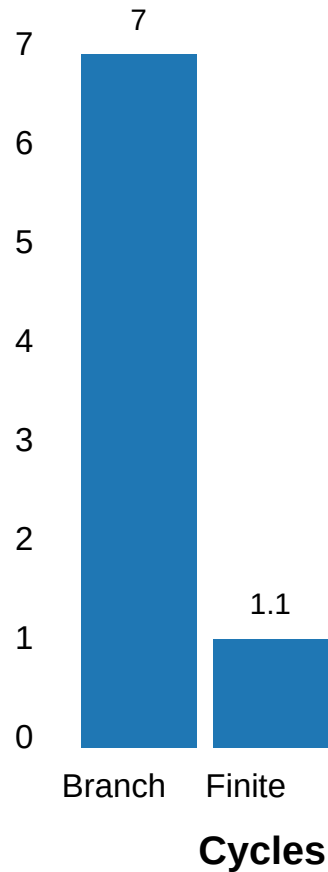
Finite state machine to the rescue

```
static uint8_t transition_table[3][256] = { {...}, {...}, {...} };

bool is_valid_utf16_ff(std::span<uint16_t> code_units) {
    uint8_t state = 0; // Start in Initial state
    for (auto code_unit : code_units) {
        uint8_t high_byte = code_unit >> 8;
        state = transition_table[state][high_byte];
    }
    return state == 0; // Valid only if we end in Initial state
}
```

Three states: default, just saw a high surrogate, error. **No branches.**

The finite-state approach can be $7\times$ faster

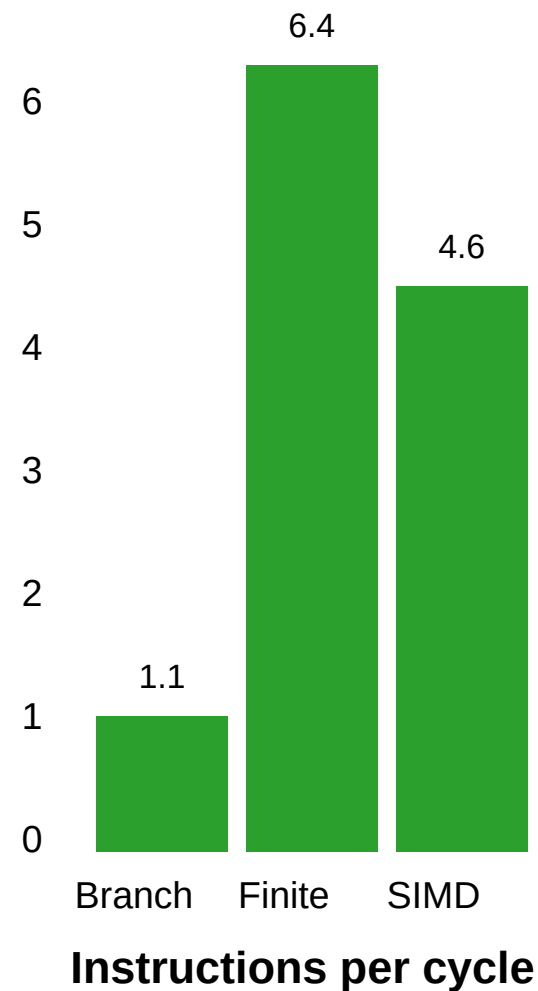
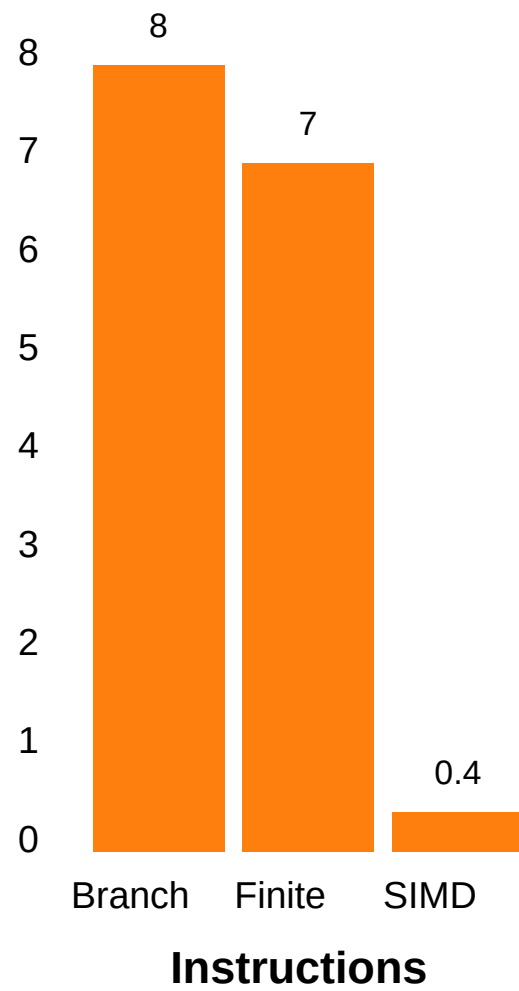
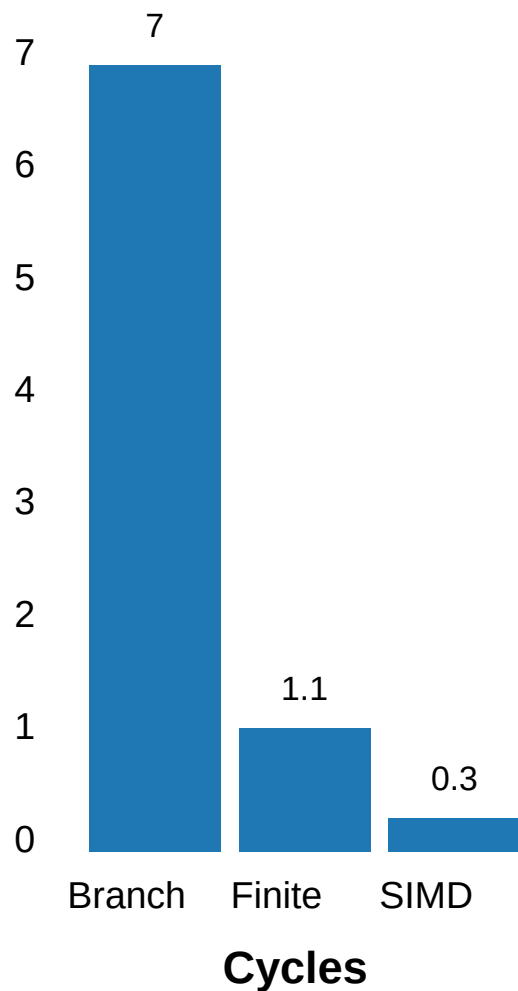


But we can do better than validation

```
const str = "ab\uD800";  
console.log(str.toWellFormed());  
// "ab?"
```

`String.prototype.toWellFormed()` must *copy and repair*, not merely check.

UTF-16, random (adversarial), Apple M4



Repair, 64 bytes at a time



2 loads, 2 ANDs, 2 compares, 1 XOR, 1 test per block: no loop-carried state, no per-unit branch
idempotent, so the tail is just one more block aligned to the end of the input

One function, every V8 embedder

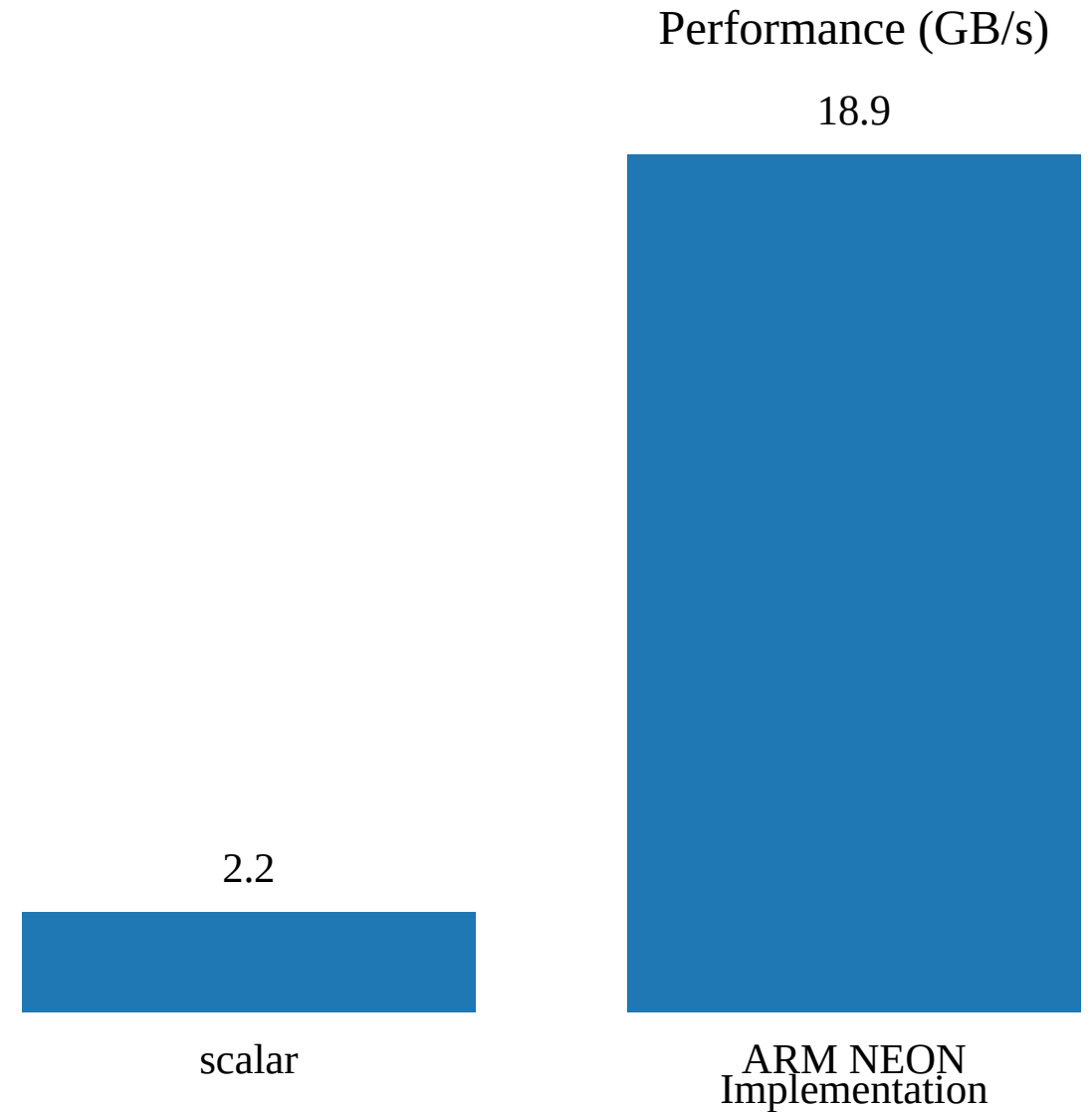


one C++ function, every V8 embedder, every well-formedness check on a JavaScript string

UTF-16 correction, Apple M4

	scalar	ARM NEON
GB/s	2.2	18.9
ins/byte	12.0	0.9

13× fewer instructions per byte.



In the browser (Apple M4)

- Chromium: 16 GB/s (**uses our new function**)
- Firefox: 3.4 GB/s
- Safari: 1.2 GB/s

Test it yourself: <https://lemire.github.io/browserwellformed/>

Case study: base64

Base64

- Encodes binary data as text using 64 characters (A-Z, a-z, 0-9, +, /)
- 3 bytes input → 4 characters output (33% overhead)
- Data URLs, email, JWTs, web APIs, embedded images
- "Hello, world!" → SGVsbG8sIFdvcmxkIQ==

Bit manipulation on a fixed schedule: **the ideal SIMD problem.**

New JavaScript functions

```
const b64 = Uint8Array.prototype.toBase64(bytes);  
const recovered = Uint8Array.fromBase64(b64);
```

function (Safari, Apple M4)	speed
<code>Uint8Array.fromBase64()</code>	11 GiB/s
<code>Uint8Array.toBase64()</code>	20 GiB/s

Test in your browser: <https://simdutf.github.io/browserbase64/>

The web's Base64 is not RFC 4648

- ES2026 `Uint8Array.fromBase64` follows WHATWG *forgiving base64*: **ignore** ASCII whitespace, **reject** anything else
- MIME e-mail breaks lines every 76 characters; DNS zone files put spaces every 50; JSON and data URLs wrap freely
- Every earlier SIMD decoder assumed clean input: one `\n` and it falls back to a byte loop

Keep the SIMD throughput on input that has whitespace in it.

Decode in blocks, compact the rare ones

64-byte block

load(in + p)



m = not alphabet

two nibble lookups, cmp



w = whitespace

only computed when m != 0



compact

vpcompressb(block, ~m)



stack buffer

append; decode when ≥ 64



64: decode this much straight to the output, slide the rest to the front

clean input

no whitespace: every block takes the fast path;
the stack buffer is never touched

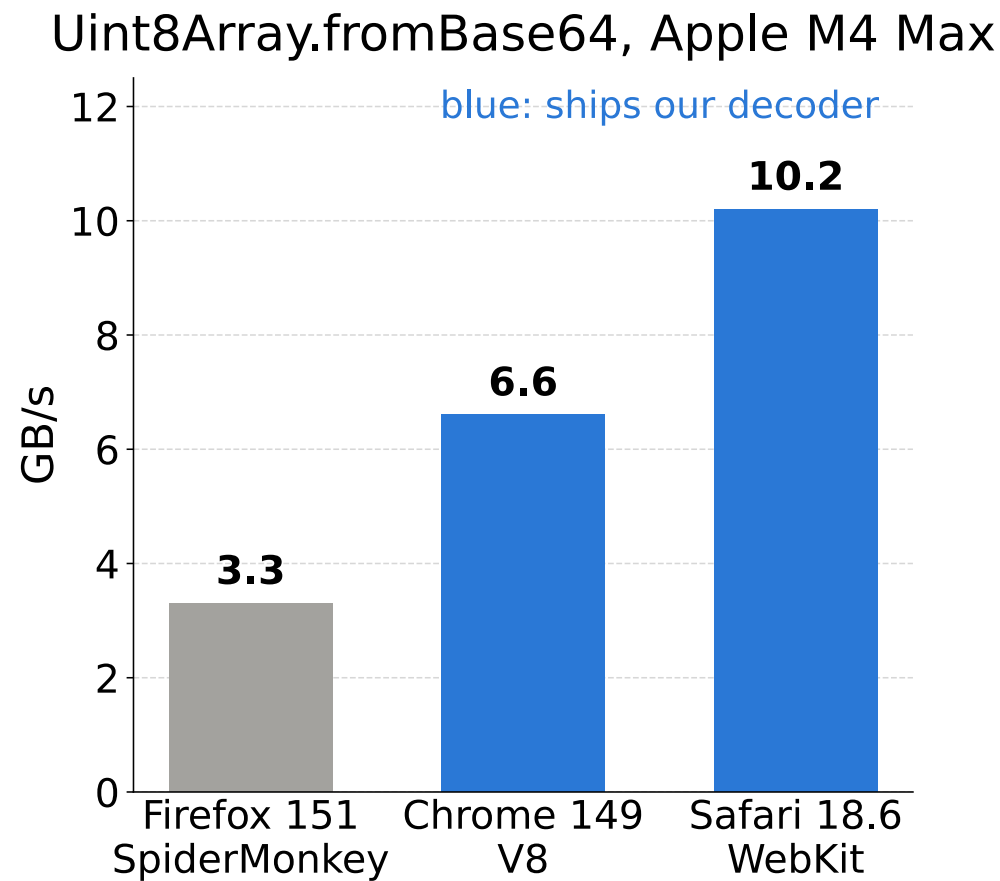
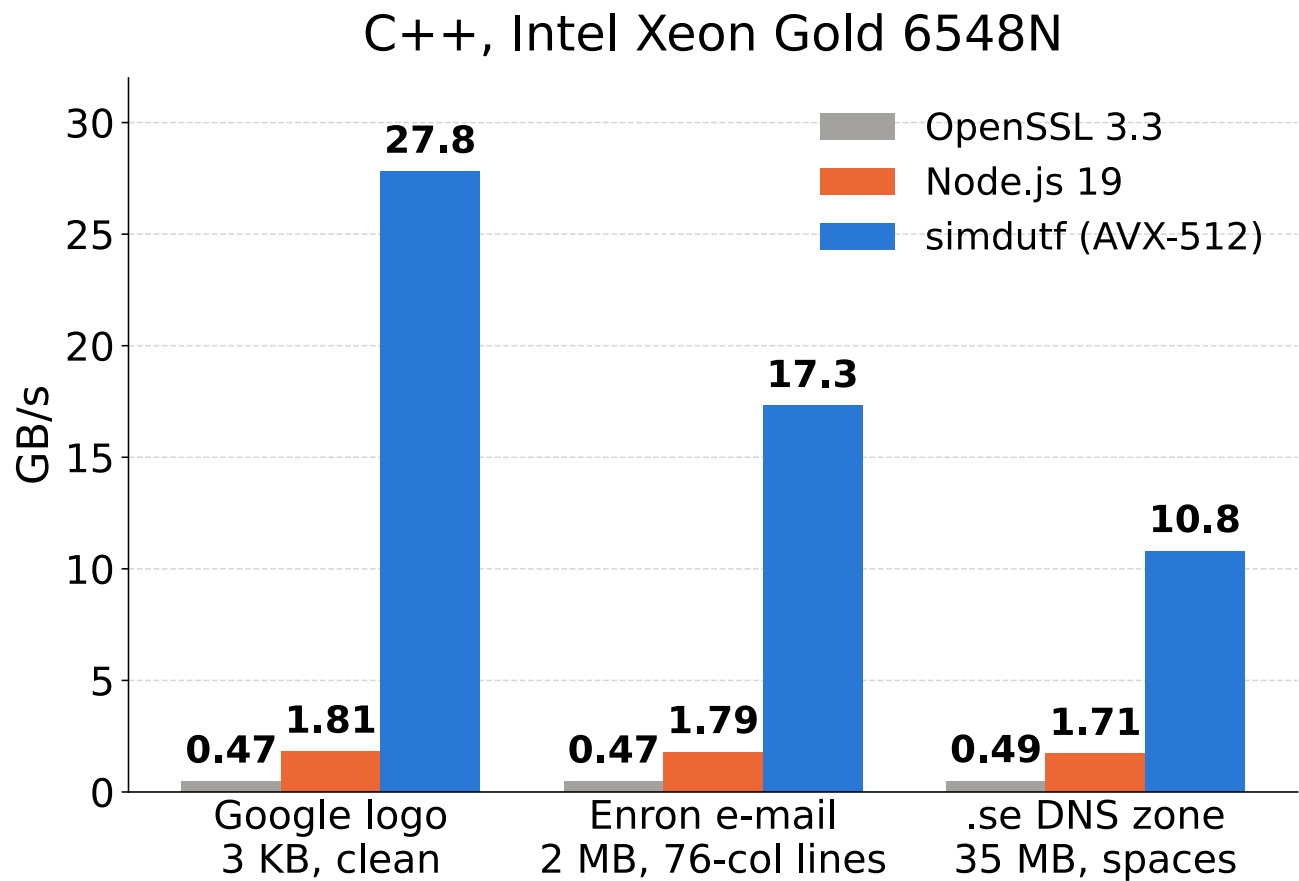
MIME / DNS / JSON

a line break every 76 chars: one compaction
per block, still SIMD throughput

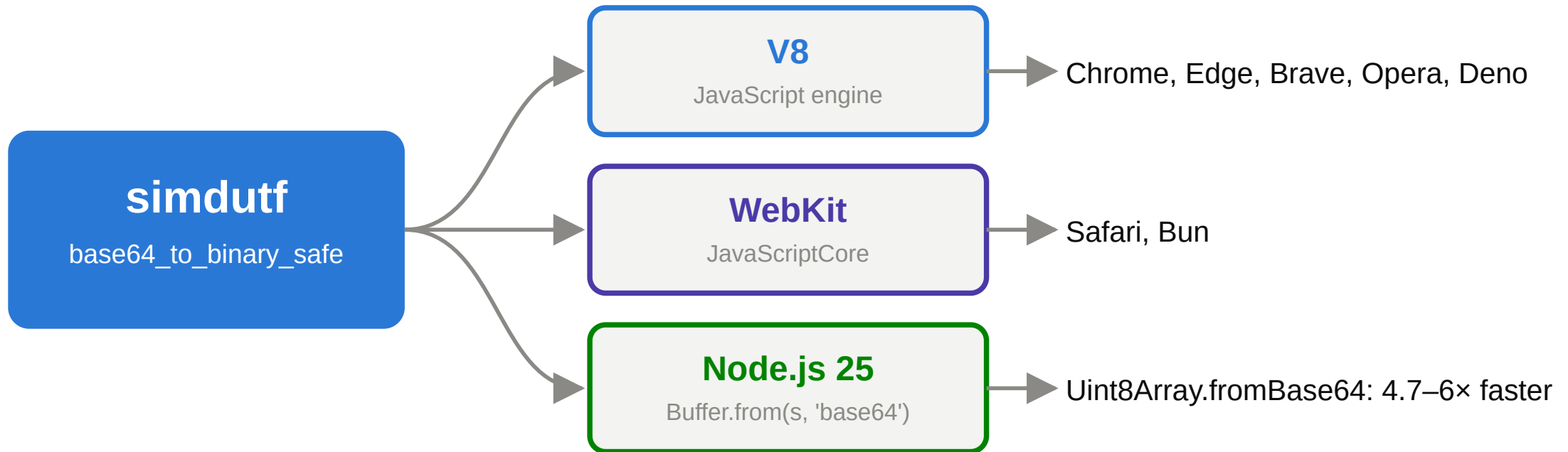
garbage

first byte that is neither alphabet nor
whitespace → error with its offset

Base64 decoding with whitespace



In the browsers, in the runtimes



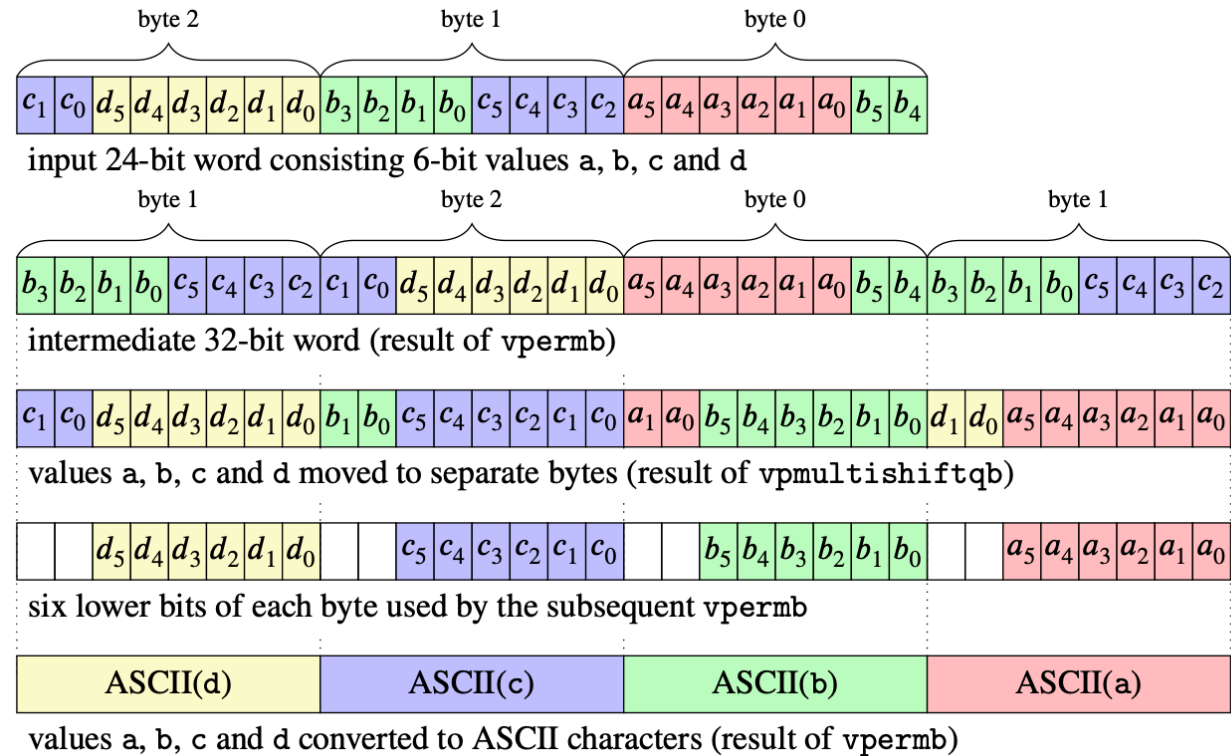
Uint8Array.fromBase64 / toBase64 landed in ES2026; the decoder underneath is this one

AVX-512 base64 encoding

- Encoding a 64-byte block requires only **two** non-memory instructions:

`vperm` (twice) and `vpmultishiftqb`.

- The right instruction turns an algorithm into a lookup.

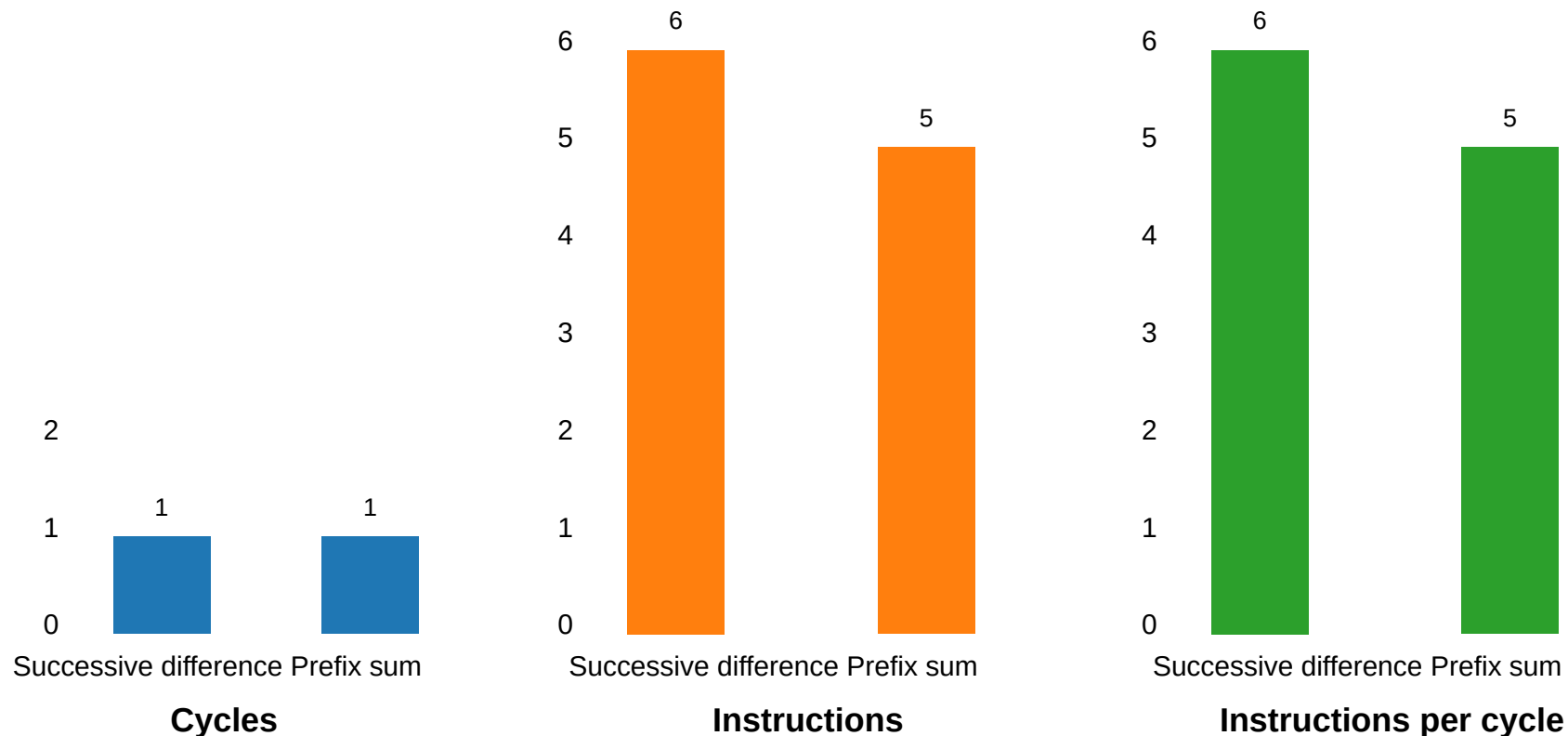


Part 5

Why your compiler will not do this for you

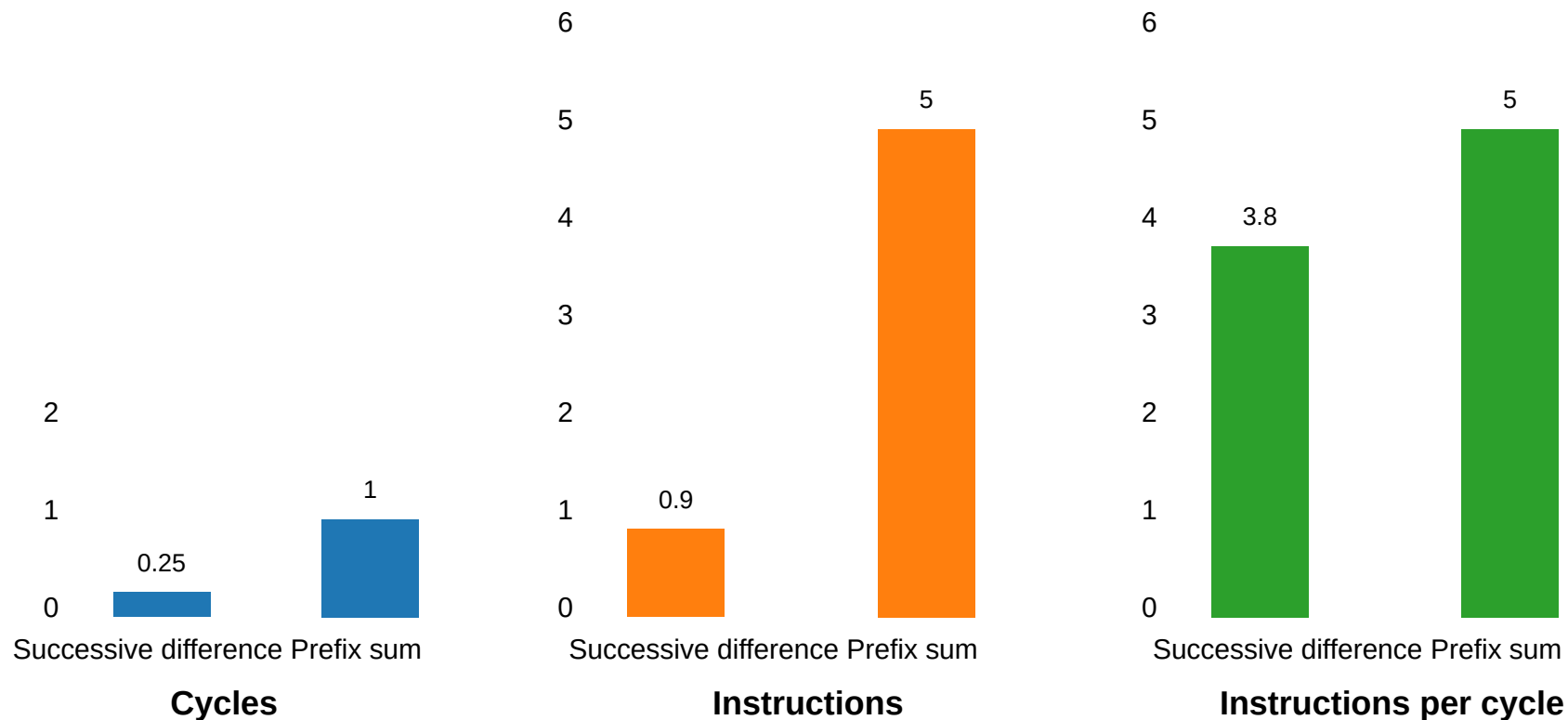
Auto-vectorization: the good case

Successive differences: `out[i] = in[i] - in[i-1]`



Scalar: **1 cycle** per element, 6 instructions per element.

Auto-vectorization: the good case



Vectorized: **0.25 cycles** per element, 0.9 instructions per element. 4× faster, for free.

Note the instructions per cycle went **down**, from 6 to 3.8.

But look at the prefix sum

- `out[i] = out[i-1] + in[i]`
- Same shape of loop. Same data. Same compiler.
- **Unchanged:** 1 cycle per element, in both builds.
- There is a loop-carried dependency, so the compiler gives up.

And yet the prefix sum *does* vectorize

- Shift-and-add, $\log_2 n$ steps: a classic parallel-prefix network.
- The compiler cannot invent it, because it is **a different algorithm**, not a different schedule.

This is the whole talk in one slide.

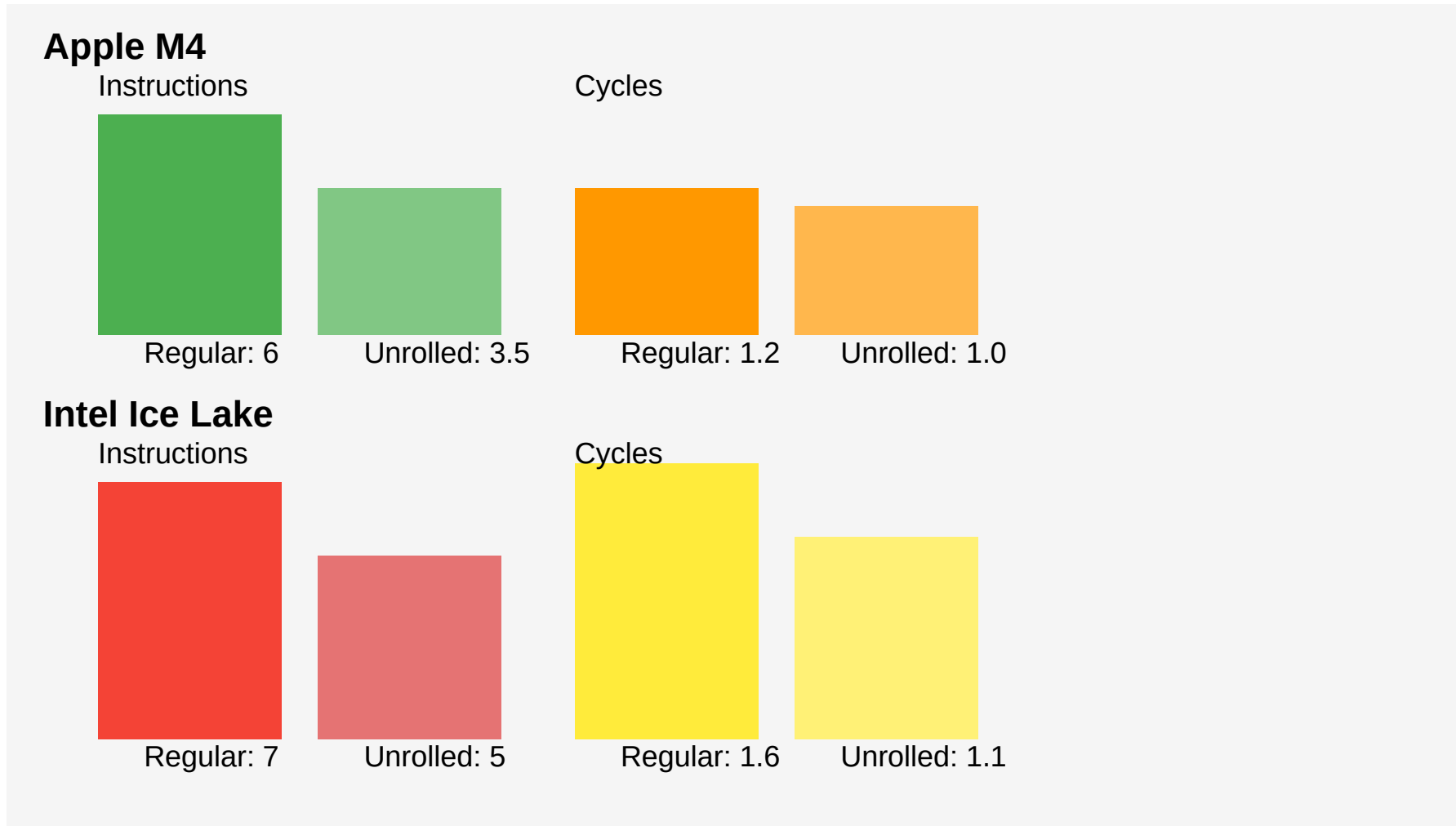
What compilers can do

- Unroll loops
- Vectorize simple, dependency-free, contiguous loops
- Choose instruction schedules

What compilers cannot do

- Change your data layout
- Change your algorithm
- Decide that a rare case can be handled on a slow path

Unrolling is not vectorizing



Fewer cycles, yes. But you are still touching one element at a time.

Part 6

Can an LLM write your SIMD code?

The honest answer: partly

Frontier models in 2026 are genuinely good at:

- Recalling intrinsic names and semantics (better than I am)
- Translating a working NEON kernel to AVX2, or to RVV
- Writing the scalar reference implementation and the test harness
- Explaining an unfamiliar instruction

Where they still struggle

- **Inventing** the vectorized algorithm (the nibble-lookup trick, the prefix network)
- Reasoning about throughput vs. latency and port pressure
- Knowing that a 512-bit instruction may downclock the core
- Noticing that the fast path is fast only on *your* data

They optimize what you measured. They do not choose what to measure.

What actually works: close the loop

Give the agent the three things it cannot produce on its own:

1. **A benchmark** it can run, over realistic inputs
2. **A differential fuzzer** against a scalar reference
3. `llvm-mca` (or `perf`) so it can see cycles, not vibes

Then let it iterate.

A workable prompt shape

Here is a scalar reference implementation and a fuzzer that compares any candidate against it.

Here is a benchmark: ``make bench`` prints cycles per byte.

Write an AVX2 version. After each attempt, run the fuzzer and the benchmark, and run `llvm-mca` on the inner loop. Do not stop until the fuzzer passes and cycles/byte is below 0.2.

The constraint is the contribution. The agent supplies the patience.

The failure mode to watch for

- An agent will happily produce SIMD code that is **correct on your test inputs** and wrong on the surrogate pair, the truncated block, the empty string, the 63-byte tail.
- Tail handling is where hand-written SIMD goes to die, and LLMs inherit that.
- **Fuzz the tails. Fuzz the alignment. Fuzz the adversarial input.**

The division of labour

you	the agent
choose the algorithm	write the intrinsics
define correctness	run the fuzzer
define the benchmark	iterate on the schedule
decide when it is fast enough	port to the other instruction sets

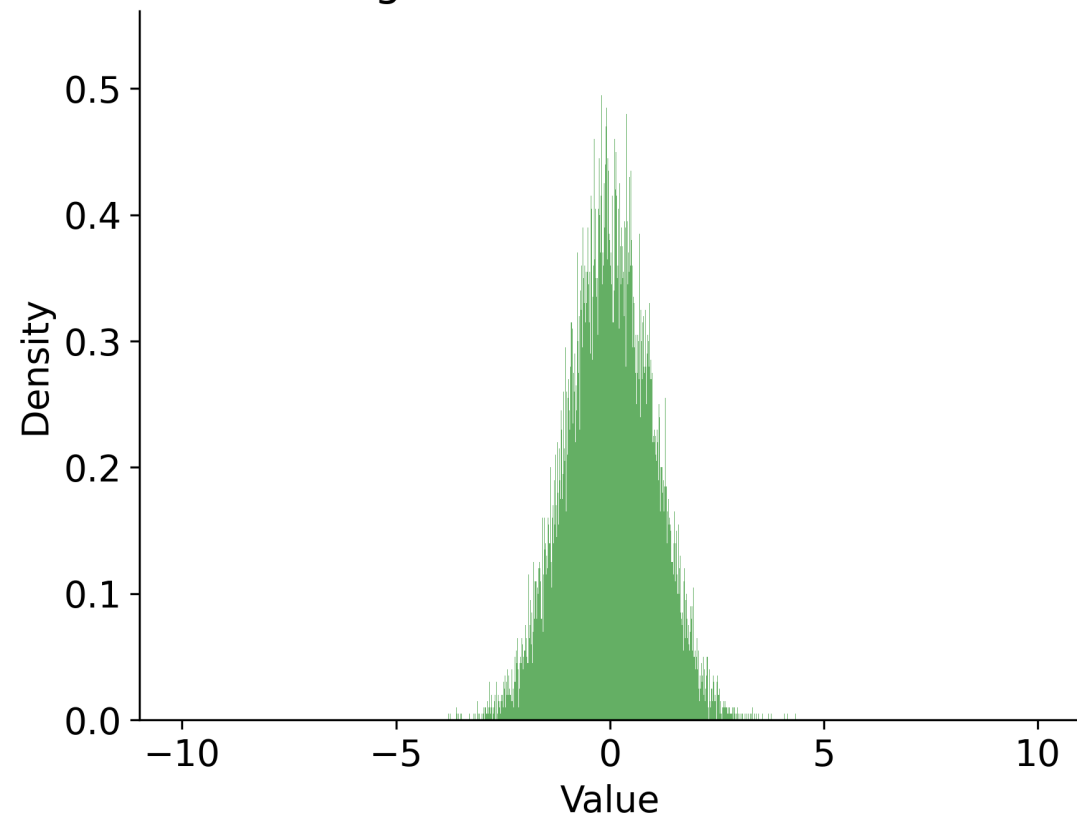
Part 7

Measure properly, or do not bother

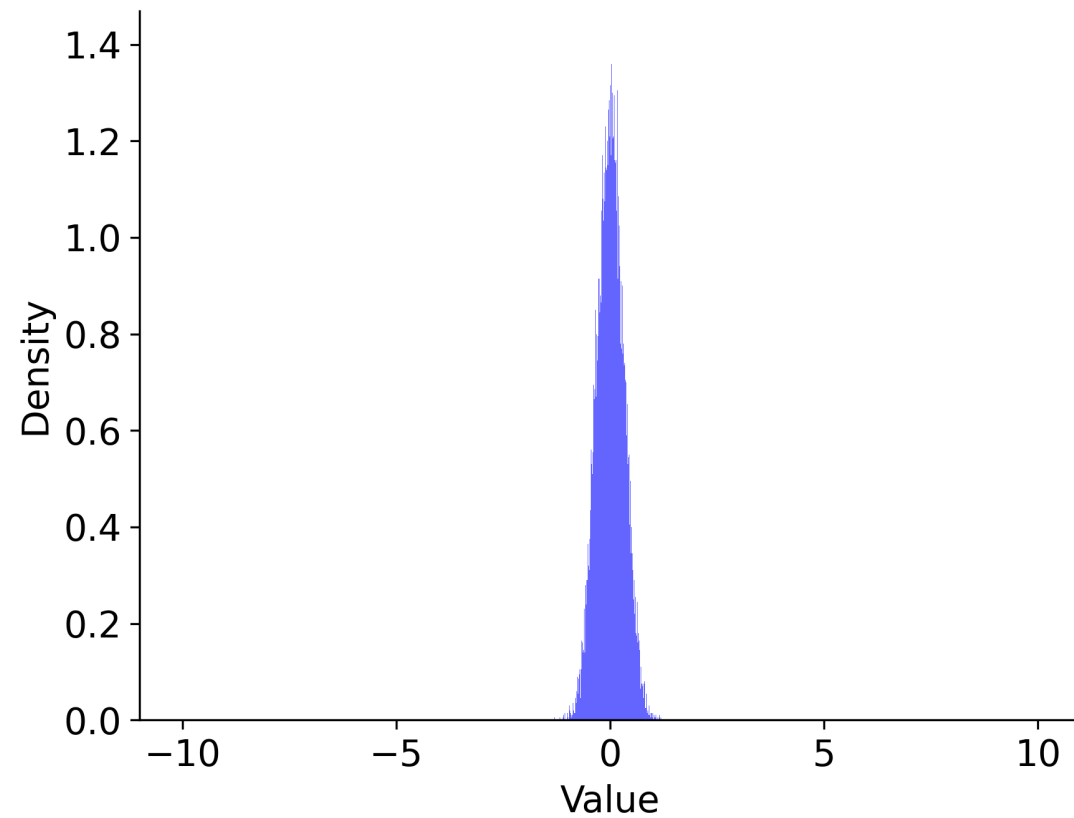
Measurements

- We often assume that measurements (timings) are normally distributed.
- If they were, the 'error' would fall off as $1/\sqrt{N}$.
- It is often an incorrect assumption.

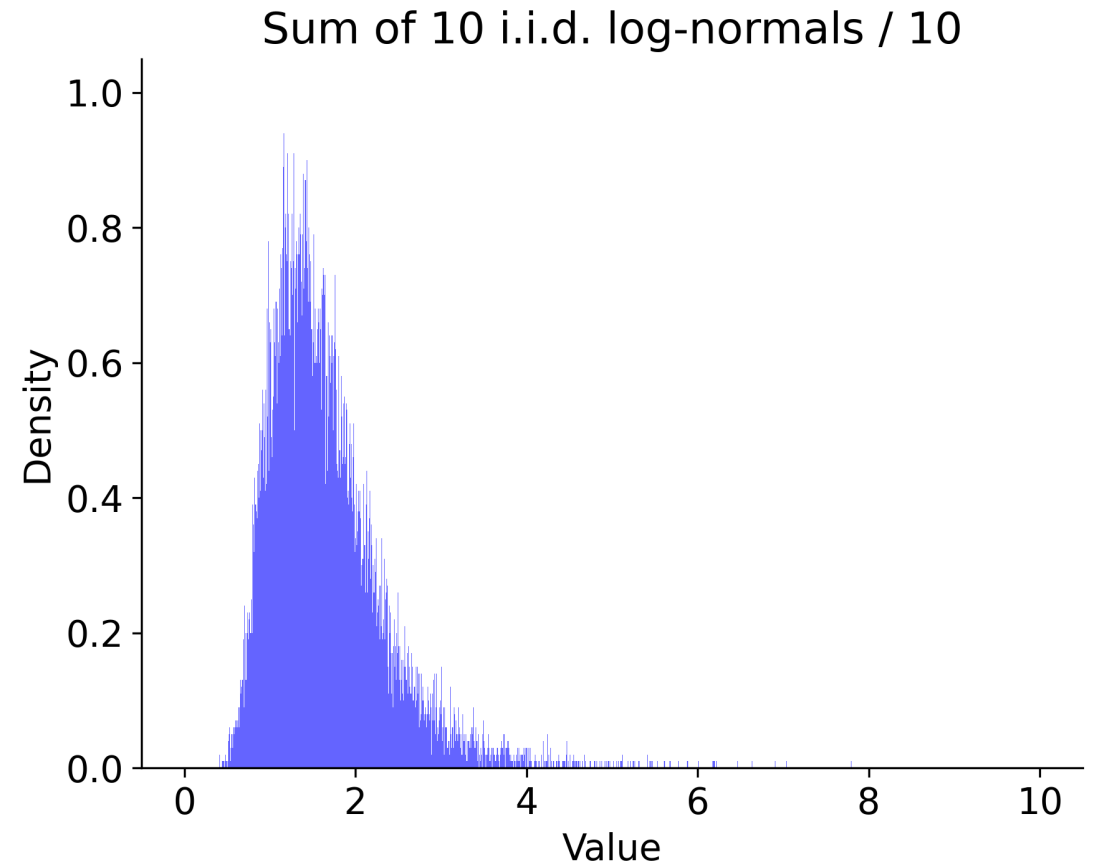
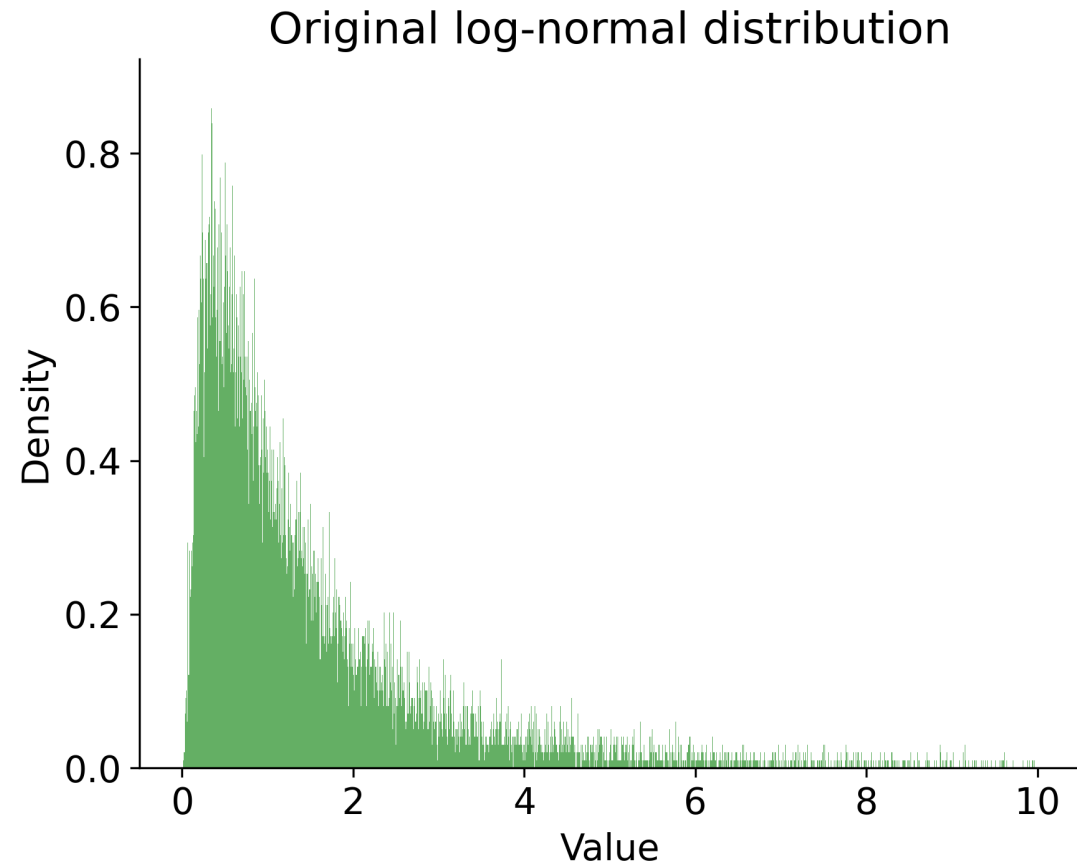
Original normal distribution



Sum of 10 i.i.d. normals / 10



What if we dealt with log-normal distributions?



Real-world measurements

- You cannot assume normality
- Measurements are **not independent**
- Reality: the absolute **minimum** is often the *reliable* metric
- Margin: the difference between the mean and the minimum

Use performance counters

Timings tell you *that* something is slow. Counters tell you *why*.

- **instructions retired** — did I actually remove work?
- **cycles** — the ground truth
- **branch misses** — is the predictor carrying me?
- **cache misses** — am I memory bound after all?

`perf stat`, `Instruments`, or a library such as `performancecounters`.

The diagnostic table I always build

variant	ns/byte	instr/byte	cycles/byte	IPC	branch miss/byte
scalar					
SWAR					
SIMD					

If instructions/byte did not drop, you did not do data parallelism.

If cycles/byte did not drop, find out which counter did move.

Hot-spot engineering fails

- A profiler shows you where the cycles are *now*.
- It does not show you the 30% of instructions spread evenly over every function.
- Reducing the total instruction count beats optimizing the top-of-profile function.

When is data parallelism worth it?

Good signs

- You touch every byte or every element
- The work per element is small and uniform
- You have an unpredictable branch per element
- You are validating, scanning, transcoding, filtering, or counting

Bad signs

- Deep pointer chasing
- Genuinely irregular control flow with expensive bodies
- You are already memory bound at full bandwidth

It applies more often than you think

- JSON parsing — every JavaScript engine
- Unicode validation — every browser
- base64 — the JavaScript standard library
- Number parsing — every compiler toolchain
- Bitmap indexes, hashing, compression, `memchr`, CSV, regex prefiltering

"Inherently serial" is usually a statement about the algorithm you happen to know.

What to take home

1. Clock speeds are flat; parallelism is where performance lives.
2. Data parallelism reduces **instructions**, not just time.
3. Your compiler schedules; it does not redesign. That part is yours.
4. Branchy code looks great in a synthetic benchmark and dies on real data.
5. IPC is a diagnostic, not a goal. The minimum time is your metric.
6. AI agents are excellent hands and mediocre architects — give them a benchmark and a fuzzer.

Interested? Check these projects

- **simdjson** — the fastest JSON parser in the world <https://simdjson.org>
 - Node.js, Bun, Deno, Electron
 - ClickHouse, WatermelonDB, Apache Doris, Meta Velox, Milvus, QuestDB, StarRocks
- **simdutf** — Unicode (UTF-8/16/32) and base64 <https://github.com/simdutf/simdutf>
 - Node.js, Bun, WebKit (Safari), Chromium (Chrome, Edge)
- **fast_float** — number parsing https://github.com/fastfloat/fast_float
- **Roaring Bitmaps** — <https://roaringbitmap.org>

Credit

- simdjson reflection work with Francisco Geiman Thiesen (Microsoft)
- simdutf UTF-16 correction is joint work with Robert Clausecker
- simdjson and simdutf are community efforts (Geoff Langdale, John Keiser, Paul Dreik, Yagiz Nizipli and others)

Questions?

Daniel Lemire — lemire.me

X: [@lemire](https://twitter.com/lemire) · GitHub: github.com/lemire

